MaiAgent 技術文件

目錄

1. 生成式 AI 快速入門

1.1 大型語言模型（LLM）

1.2 RAG 知識庫檢索系統

1.3 Embedding 模型

1.4 Reranker 模型

1.5 Parser 解析工具

1.6 圖像辨識支援

2. 技術原理

2.1 角色指令

2.2 回覆評估與監測結果

2.3 Text to SQL

2.4 Function Calling

2.5 Skill 技能模組

2.6 Canvas

2.7 AI 安全防護機制

2.8 OAuth 2.0 整合與 Token 自動刷新機制

2.9 ICAP 內容驗證整合

3. MaiAgent Work｜員工生產力整合

3.1 把 MaiGPT 加進您的系統

3.2 串接 MaiGPT：選對 Agent、補齊能力

4. MaiAgent Serve｜客戶服務整合

4.1 軟體廠商整合指南：把 AI Agent 嵌給您的客戶

4.2 Web Chat 嵌入與 SDK

4.3 MaiGPT 模式嵌入

4.4 注入頁面 Context

4.5 LINE LIFF：什麼是 LINE LIFF

4.6 LINE LIFF：如何串接

4.7 聯絡人 (Contact) 介紹及串接

4.8 聯絡人身分同步與 Token 更新

4.9 角色(Role) 和 聯絡人(Contact) 的差異

4.10 第三方客服系統串接（含轉真人）

4.11 Genesys Cloud 整合

4.12 對話交接與接續對話 API

4.13 Webhook

5. MaiAgent Build｜API 串接指南

5.1 快速上手

5.2 API 共通規範

5.3 建立對話與訊息

5.4 對話與訊息回覆（串流/同步）

5.5 Presigned 檔案上傳模式

5.6 使用知識庫—上傳、搜尋與管理

5.7 API 工具工程指南

5.8 n8n 串接

5.9 Google Sheet 串接

5.10 端點行為補充

5.11 對話附件管理 API

5.12 對話產出清單 API

5.13 對話可見權限與即時事件

5.14 成員記憶管理 API

5.15 站台品牌設定 API

5.16 Credits 項目顯示順序

5.17 Agent 回答模式與結構化輸出欄位

5.18 對話記錄的用量與 Credits 欄位

5.19 內建工作流回覆模式退場

6. MaiAgent Build｜MCP 與外部服務

6.1 MCP（Model Context Protocol）

6.2 MaiAgent MCP：讓外部 AI 工具操作平台

6.3 自架 MCP Server 接入規格

6.4 Remote MCP 服務概述

6.5 Composio 串接

6.6 Zapier 串接

7. MaiAgent Build｜AI Gateway

7.1 AI Gateway：OpenAI 相容 API 串接

7.2 AI Gateway 安全性

8. 身分與權限

8.1 成員開通：SCIM／邀請／SSO

8.2 即時同步：在使用者事件中呼叫

8.3 名單同步工具：CSV／JSON／資料庫

8.4 SSO 與身分整合總覽

8.5 SAML SSO 單一登入整合

8.6 MaiAgent 作為 OAuth 授權伺服器

8.7 知識管理權限(Query Metadata / 查詢元資料) 總覽

8.8 開始建構—使用 JSON 格式

9. 平台建置與部署

9.1 概覽

9.2 平台環境（SaaS/私有雲/地端）

9.3 部署架構

9.4 部署型態與功能對照

9.5 系統需求

9.6 版本與升級

9.7 地端模型服務接入

9.8 維運重點：離線安裝、TLS、備份、監控與驗收

9.9 自帶向量資料庫

9.10 雲端模型推論 API 服務

9.11 GPU 算力硬體規劃

9.12 檔案庫 Office 線上編輯（ONLYOFFICE Docs 整合）

生成式 AI 快速入門

大型語言模型（LLM）

選擇的關鍵點

在選擇大型語言模型時，請考慮以下幾個關鍵因素：

環境（Environment）：使用環境是否能夠上網來決定使用雲端或地端模型。
品質（Quality）：模型生成回覆結果的能力與對於指令的服從性。
速度（Speed）：文字生成速度及對於延遲的要求，來確保模型的回應速度。
價格（Pricing）：各模型的 Credits 計價不同，請依使用需求與預算選擇，計價方式見 Credits 計費制度。
其他：是否支援多模態、是否支援 Function calling。

各模型的品質、速度與價格比較，可參考 Artificial Analysis 等持續更新的第三方評測。

MaiAgent 上支援的大型語言模型

MaiAgent 上架的模型會隨供應商發布與退役持續更新，本頁只列供應商與模型家族，不列具體型號。組織實際可選用的模型，以管理後台的「模型調用權限」頁為準：該頁列出 MaiAgent 為組織開通的模型，組織擁有者可在範圍內自行啟用或關閉。

閉源模型

模型供應商	模型家族	推論服務
OpenAI	GPT、o 系列	OpenAI、Azure、AWS Bedrock
Anthropic	Claude	AWS Bedrock、Google Vertex AI
Google	Gemini	Google Vertex AI
xAI	Grok	OpenAI 相容 API
Alibaba	Qwen（Max、Plus）	OpenAI 相容 API
Amazon	Nova	AWS Bedrock

開源模型

開放權重（open-weight）模型可以在 MaiAgent SaaS 上透過雲端推論服務使用，也可以在私有雲、地端以自建 GPU 部署，所需硬體請參考 GPU 章節。

模型供應商	模型家族	SaaS 上的推論服務
OpenAI	GPT-OSS	OpenAI 相容 API
Google	Gemma	OpenAI 相容 API
Alibaba	Qwen、Qwen-VL	OpenAI 相容 API
DeepSeek	DeepSeek	AWS Bedrock、Azure
Mistral AI	Mistral	OpenAI 相容 API
Moonshot AI	Kimi	OpenAI 相容 API
IBM	Granite	OpenAI 相容 API

地端自建時可部署的模型，以 MaiAgent 部署包的模型目錄為準，目前包含 GPT-OSS、Gemma、Qwen、Qwen-VL 等家族；部署規劃請洽 MaiAgent 窗口。

> 各供應商類型支援的 LLM、Embedding、Reranker 範圍不同，詳見模型 API 推論服務。

模型一定要 Fine-tune 嗎？

隨著人工智慧技術的快速發展，語言模型已經擁有了強大的語言理解和生成能力，並在許多領域中廣泛應用。例如，預訓練的語言模型可以輕鬆應對日常對話、文章生成以及簡單的問答任務。

然而，當模型面對更具挑戰性的專業領域任務，如醫療、法律或技術支持等領域時，僅僅依賴預訓練模型可能無法提供最佳的表現。有些開發者會選擇進行 **Fine-tuning**，也就是針對特定領域進行額外的訓練，以提高模型的專業知識。

然而，Fine-tuning 並不是唯一的解決方案，還有其他兩種有效的方法可以達成這一目標：**Prompt Engineering** 和 **RAG (Retrieval-Augmented Generation)**。

1. Prompt Engineering：通過精確的提示優化模型表現

Prompt Engineering 為設計精確的提示語句來引導模型生成所需的結果。這種方法的核心是根據任務需求，設計詳細表達方式，幫助模型縮小回答範圍，理解上下文和所需的輸出形式。

假設有一個語言模型，目的是根據用戶的需求推薦合適的產品。在這個情況下，如果表達 「我想買一部高性能的手機」，可能會導致模型生成不夠精準的答案，因為「高性能」這個詞有很多不同的解釋，可能指的是處理速度、相機性能、電池壽命等。

為了提高推薦的精度，我們可以進行 **Prompt Engineering**，設計更加具體的問題或提供額外的上下文來引導模型更好地理解用戶的需求。例如，改為以下的提示語句：

"我需要一部具備**長時間電池續航**和**高效處理器**的手機，並且價格在 **500 美元到 800 美元**之間，請推薦幾款符合條件的手機。"

2. RAG：結合外部知識提升生成能力

RAG 是通過將外部知識檢索和生成過程相結合，來提升模型的表現。

在傳統的生成任務中，模型僅依賴預訓練時學到的知識。而 RAG 則利用檢索系統，即時獲取外部資料，並將這些資料與生成模型結合，從而更準確地回答問題或生成文本。

舉個例子，在醫療領域，當模型被問及某種罕見疾病時，RAG 可以先從專業醫學資料庫中檢索相關資料，再根據這些資料生成更準確的回答。這種方法的好處是，即便模型本身未在訓練過程中見過某些資料，它仍然能夠通過檢索現有知識來做出高質量的回應，特別適用於需要即時知識更新的情境，並能大大擴展模型的知識範圍。

> 有關 RAG 更詳細的介紹，將於下一章 「RAG 知識庫檢索系統 說明」。

總結，Fine-tuning、Prompt Engineering 和 RAG 各有其優勢和適用範圍，可根據應用場景和需求選擇最合適的策略，而不必僅依賴於單一的 Fine-tuning 方法。

Prompt Engineering 提供了一種成本低且靈活的解決方案，通過設計精確的提示語句來引導模型產生高質量的結果。

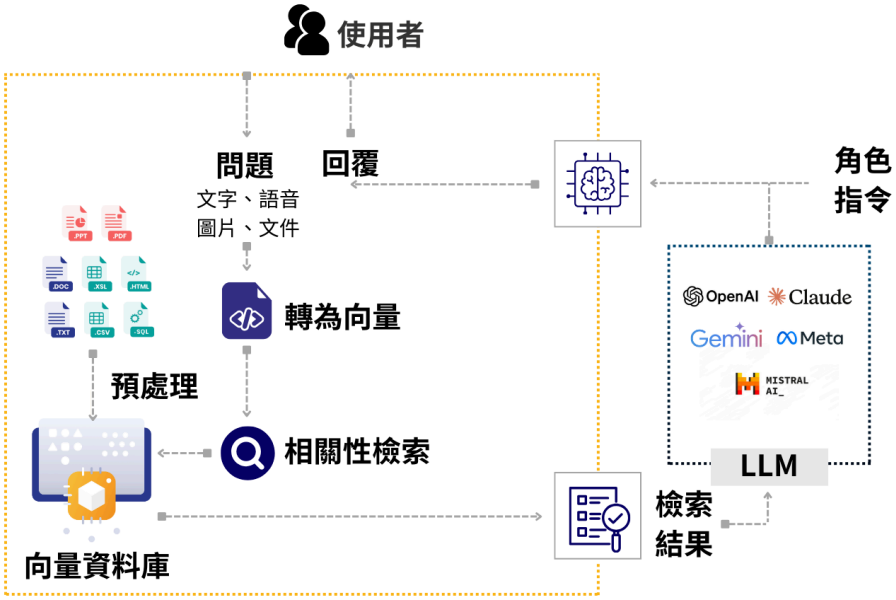
而 **RAG** 則提供了一種結合外部知識和生成能力的方法，能夠在需要動態獲取知識的情況下，提供更精確的答案。

Fine-tuning 可以顯著提升模型在特定領域的表現，但需要大量專業資料並消耗額外資源，可視為 **Prompt Engineering** 與 **RAG** 兩種方式都無法成功，才實行的最後手段

> MaiAgent 平台不提供模型微調功能：後台沒有訓練或微調模型的介面，平台也不會用您的資料訓練任何模型。上面提到的 Fine-tuning 是模型供應商或自建模型團隊在平台之外完成的事。已經微調好的模型可以這樣接進平台：雲端供應商託管的微調模型，以自帶金鑰串接（見 AI Gateway：OpenAI 相容 API 串接）；自己訓練的開放權重模型，在私有雲或地端部署成 OpenAI 相容的模型服務後接入，見地端模型服務接入。要讓 Agent 掌握專業知識，請優先用角色指令與 RAG 知識庫。

RAG 知識庫檢索系統

RAG（Retrieval-Augmented Generation，檢索增強生成）讓大型語言模型在回答前，先從知識庫找出與問題相關的段落，再依這些段落作答。模型因此能引用組織自己的文件，而不只依賴訓練資料。



RAG 流程

可以把 RAG 想成開書考試：模型不必背下所有資料，而是每次作答前先翻知識庫，找出和題目最相關的段落當作依據。也因為如此，知識庫只放與 Agent 任務相關的資料比較好；不相關的內容仍可能被檢索出來當成上下文，反而讓回答失準。

本頁說明 MaiAgent 實際的處理流程與可調整的參數。後台操作步驟見使用者手冊的 知識庫總覽。

資料如何進入知識庫

上傳到知識庫的檔案會依序經過：

解析：解析器把檔案轉成文字（必要時含表格與圖片），見 Parser 解析工具。

切塊：文字切成段落（chunk），之後檢索與引用都以段落為單位。

向量化：以知識庫選定的 Embedding 模型把每個段落轉成向量，見 Embedding 模型。

寫入向量資料庫：段落文字與向量一起寫入向量資料庫，預設使用平台內建的資料庫；也可以改接自己的資料庫，見 自帶向量資料庫。

檔案狀態變成 `done` 後，內容才會被檢索到。

一次檢索的流程

1. 決定檢索範圍

檢索前，平台先決定這次可以查哪些知識庫、檔案與標籤，並把條件直接加在向量資料庫的查詢上。不在範圍內的段落根本不會被查出來，也就不會進入模型。

範圍依序取自：訊息 → 對話 → 聯絡人的查詢元資料（Query Metadata）；三者都沒有設定時，套用 Agent 的預設標籤過濾；都沒有時查詢整個知識庫。設定方式見 Query Metadata 總覽 與使用者手冊的 知識庫標籤過濾。

> 要限制「誰能看到哪些文件」，請用查詢元資料或標籤過濾，不要只靠角色指令。角色指令是在檢索之後才給模型的文字，擋不住已經被檢索出來的段落。

2. 混合檢索

預設以**混合檢索**同時進行兩種查詢：

向量檢索：用問題的向量找語意相近的段落，措辭不同也找得到。
關鍵字檢索（BM25）：比對字詞，適合產品型號、專有名詞、代碼。

知識庫開啟 啟用 Reciprocal Rank Fusion（RRF）時，兩種結果以 RRF 依排名合併。在 Elasticsearch 上使用 RRF 需要 Enterprise、Platinum 或 Trial 授權；自帶的 Elasticsearch 沒有這類授權時，後台會自動關閉這個選項。平台關閉混合檢索時，改為只用向量檢索。

Agent 對話時一律依平台設定檢索，不能另選模式。後台的 搜尋測試 與 知識庫檢索 API 則可以另外指定只用向量（語意搜尋，`semantic`）或只用關鍵字（關鍵字搜尋，`keyword`，僅 Elasticsearch），方便比對兩種檢索各自找到什麼。

3. Reranker 重新排序

知識庫開啟 啟用搜尋結果重排序 (Reranking) 時，第一階段會先多取一些候選段落（檢索片段數量 × 平台設定的放大係數，預設 7 倍），再由 Reranker 模型逐一比對問題與段落、重新評分，只保留分數最高的 檢索片段數量 筆。說明見 Reranker 模型。

組織 Credits 低於平台門檻時，該次對話會略過 Reranker，直接使用第一階段的結果。

4. 相似度門檻

相似度門檻值（0-1） 會濾掉 Reranker 評分低於門檻的段落。預設為 0，代表不過濾。門檻作用在 Reranker 的分數上，所以**只有開啟 Reranker 時才有效**。設得太高時，可能一筆段落都不剩，模型就沒有參考資料可用。

5. 多個知識庫的合併

Agent 掛了多個知識庫時，各知識庫的結果會合併，再依 Agent 的 檢索區塊最大數量 決定最後送進模型的段落數。只要這次查到的知識庫中有一個開啟 Reranker，合併後的結果就會再經過一次重新排序。

6. 送進模型並標示引用

選出的段落連同 Agent 的角色指令一起送進大型語言模型。檔案的中繼資料欄位勾選 檢索時使用 時，該欄位的值也會一併送進模型，作為回答判斷的參考。回覆中的引用會連回來源段落與檔案。

Agentic RAG：Agent 模式下的檢索

上面的流程是「一次檢索、一次生成」：每則訊息固定先檢索再回答，模型不決定要不要查、查什麼。Agentic RAG 則把檢索交給模型決定：模型在回答過程中可以多次呼叫檢索，依前一次結果調整查詢、換關鍵字或改讀另一份檔案，再整理成回答。

在 MaiAgent，兩者對應 Agent 的回答模式：

回答模式	檢索方式
RAG 問答（API 的 `replyMode: normal`）	一次檢索：每則訊息依本頁流程檢索一次，段落直接送進模型
Agent 模式（`replyMode: agent`）	Agentic RAG：知識庫以工具的形式交給模型，模型可在同一回合內多次檢索段落、依檔名或中繼資料篩選檔案、以關鍵字搜尋檔案內容、讀取整份檔案，每次呼叫都走本頁的檢索流程（包含 Query Metadata 的範圍限制）

Agent 模式的檢索輪數受 Agent 的執行上限約束（見 Function Calling：執行上限），延遲與用量也比一次檢索高。問答型、答案在單一段落的場景用 RAG 問答即可；需要比對多份文件、先找檔再細讀的場景再用 Agent 模式。

對話模式與 Agent 模式怎麼查知識庫

承上節，兩種模式的差異逐項對照如下：

	RAG 問答（對話模式，`replyMode: normal`）	Agent 模式（`replyMode: agent`）
何時檢索	每一則訊息回答前**固定**先檢索一次，不經模型判斷	檢索是**工具**，模型依問題決定要不要呼叫、呼叫幾次、用什麼查詢字串

	RAG 問答（對話模式，`replyMode: normal`）	Agent 模式（`replyMode: agent`）
取幾筆	每個知識庫各取自己的檢索片段數量，合併後依 Agent 的檢索區塊最大數量截斷	`retrieve_text_nodes` 工具由模型自填 `top_k`（預設 3），所以通常少於檢索片段數量；Agent 勾選強制使用完整檢索範圍時，一律改用檢索區塊最大數量
可用的工具	無，只有這次檢索到的段落	`retrieve_text_nodes`（語意／混合檢索）、`query_files`（列出檔案與中繼資料）、`grep_files`（在檔案內容比對字串）、`read_file`（整份讀取）、`get_surrounding_context`（取段落前後文）、`image_search`（多模態知識庫找圖，可在工具設定關閉）
查詢元資料、標籤過濾、Reranker、相似度門檻	兩種模式都套用，規則相同	
對話中上傳的附件	上傳時立即解析、建索引，回答時與知識庫一起檢索，取 Agent 的檢索區塊最大數量	上傳時不解析；Agent 依問題用附件工具（`parse_attachment_content`、`read_attachment_full_text`、`ingest_attachment`）讀取或建索引，見 訊息附件怎麼被處理
搜尋結果不理想時	調整知識庫參數與 Agent 的檢索區塊最大數量	先確認模型有沒有呼叫檢索工具（在對話詳情與執行時間軸看工具呼叫），再調參數或在角色指令要求先查知識庫

Agent 開啟啟用工具搜尋功能（動態工具搜尋）時，知識庫工具也在「要先搜尋才載入」的池子裡，和 Text to SQL、API 工具一樣要先呼叫 tool_searching_tool。模型偶爾會跳過搜尋、回答「沒有知識庫檢索工具」而只用其他工具；遇到時在角色指令寫明「回答前先用 retrieve_text_nodes 查詢知識庫」，或關閉工具搜尋讓所有工具直接可用。掛載知識庫本身不會自動檢索，也不需要另外建立檢索工具。

可調整的參數

參數	設定位置	預設值	說明
Embedding 模型	知識庫	依平台設定	建立後無法在後台更改；需要更換時請另建知識庫，或洽 MaiAgent 窗口
檢索片段數量	知識庫	12	每個知識庫最多取回的段落數，最小值 1
啟用搜尋結果重排序 (Reranking)	知識庫	開啟	開啟後使用知識庫選定的 Reranker 模型
相似度門檻值（0-1）	知識庫	0	只在開啟 Reranker 時生效

參數	設定位置	預設值	說明
啟用 Reciprocal Rank Fusion	知識庫	依平台設定	混合檢索的合併方式；OpenSearch 的混合檢索必須開啟
檢索區塊最大數量	Agent	12	對話時最後送進模型的段落數；介面建議 5～15，數值越高延遲越高

想確認某個問題會檢索到哪些段落，可以使用後台的 搜尋測試，或呼叫 知識庫檢索 API。

怎麼調

片段大小與重疊由解析器決定，後台沒有欄位可調；能調的是下列參數。每次只改一項，用同一組問題在搜尋測試比較。

症狀	先檢查	再調整
正確段落找得到，但排在很後面或被截掉	搜尋測試中它排第幾	開啟 Reranker；仍不夠時提高檢索片段數量與檢索區塊最大數量（延遲與 token 會增加）
常帶進不相關的段落	開啟 Reranker 後各段落的順序	調高相似度門檻值（每次加 0.05～0.1，太高會一筆都不剩）；或用標籤過濾縮小範圍
型號、代碼、專有名詞找不到	用關鍵字搜尋能不能找到	確認文件中的寫法和使用者一致；混合檢索關閉時改回開啟（Elasticsearch 需 RRF 授權）
表格或長文件只答到一半	片段是否把表格切開	換解析器重新解析、在片段內補上表頭或章節名稱，或在 Agent 勾選強制使用完整檢索範圍
回答只用到附件、沒用知識庫（或相反）	Agent 模式下工具呼叫記錄	在角色指令寫明先查知識庫；對話模式下兩者會一起檢索，提高檢索區塊最大數量
Reranker 開了但沒效果	組織 Credits 是否低於平台門檻（會自動略過 Reranker）	先儲值，再比較

Embedding 模型建立後不能換，換模型等於重建知識庫；請在上線前用搜尋測試比較不同模型的結果再定案。

Embedding 模型

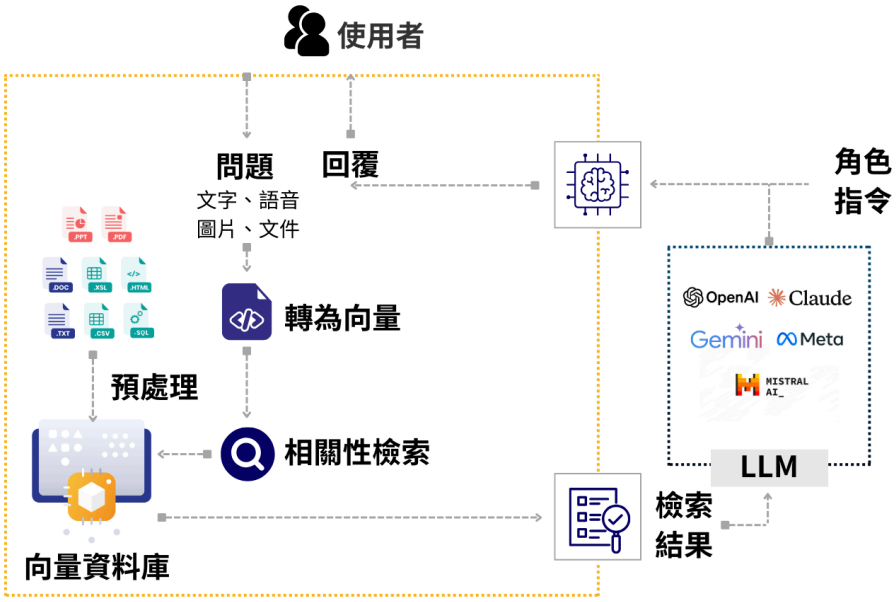
什麼是 Embedding？

Embedding 模型把文字轉成一串數字（向量）。語意相近的文字，轉出來的向量也相近，所以系統可以用向量距離找出「意思相關」的段落，即使問題和文件的措辭不同。

在 MaiAgent 的 RAG 流程中，Embedding 模型做兩件事：

建立索引：把知識庫解析後的每個段落轉成向量，存進向量資料庫。

轉換問題：把使用者的問題轉成向量，和資料庫中的段落比對相似度。



RAG 流程

完整的檢索流程見 RAG 知識庫檢索系統。

選擇時要知道的事

建立後無法在後台更改：Embedding 模型在建立知識庫時選定。不同模型產生的向量無法互相比對，需要更換時請另建知識庫，或洽 MaiAgent 窗口。

圖片檔需要多模態模型：要把圖片檔（例如 `.png`、`.jpg`）放進知識庫並建立索引，知識庫必須使用支援多模態的 Embedding 模型，否則該檔案會解析失敗。

Agent 可以混用：同一個 Agent 可以掛使用不同 Embedding 模型的知識庫，平台會各自查詢後合併結果。

MaiAgent 雲端啟用中的模型

以下為 2026 年 10 月正式區的啟用清單，建立知識庫時可選的模型以後台顯示為準。

模型（後台名稱）	供應商	支援圖片檔
OpenAI text-embedding-3	OpenAI（`text-embedding-3-large`）	否
Cohere Embed V4.0	Cohere（`embed-v4.0`）	是
Cohere Embed Multilingual v3.0 (Bedrock)	AWS Bedrock（`cohere.embed-multilingual-v3`）	否

地端部署

地端與私有雲可以在本機部署 Embedding 模型（部署包的模型目錄提供 `jina-embeddings-v3`，需要 GPU），或接上既有的模型服務、雲端供應商。可用的模型與接法見 地端模型服務接入；各雲端供應商是否支援 Embedding 見 雲端模型推論 API 服務。

自備 Embedding 服務的介面要求

接上自行架設或第三方的 OpenAI 相容 Embedding 服務時，網址要包含 `/v1`，服務也必須接受 MaiAgent 送出的參數，否則知識庫檔案會全部解析失敗。完整規則見 地端模型服務接入：自備 Embedding 與 Reranker 的限制。

> Embedding 模型影響的是「找不找得到」。若某個問題常常找不到正確段落，可以先用後台的 搜尋測試 檢查結果，再評估調整 Reranker、檢索片段數量，或改用其他 Embedding 模型重建知識庫。

Reranker 模型

什麼是 Reranker？

Reranker 是專門做「重新排序」的模型。第一階段的檢索（向量與關鍵字）速度快，但只能粗略比對；Reranker 把問題和每一個候選段落放在一起逐一判斷相關程度，再依分數重新排列，讓真正回答問題的段落排在前面。

初步檢索

以向量與關鍵字從知識庫快速取回一批候選段落。

重新評分

Reranker 逐一比對「問題＋段落」，給每個段落一個相關分數。

篩選

依分數保留前幾名，並濾掉低於相似度門檻的段落，再交給大型語言模型。

MaiAgent 如何使用 Reranker

每個知識庫各自選定一個 Reranker 模型，並以 啟用搜尋結果重排序 (Reranking) 開關（預設開啟）。

開啟時，第一階段會取回 檢索片段數量 × 平台設定的放大係數（預設 7 倍）筆候選，Reranker 重新排序後只保留 檢索片段數量 筆。

相似度門檻值（0-1） 作用在 Reranker 的分數上，所以只有開啟 Reranker 時才有效。

關閉時，第一階段直接取回 檢索片段數量 筆，不做重新排序，也不套用相似度門檻。

完整流程見 RAG 知識庫檢索系統。

MaiAgent 雲端啟用中的模型

以下為 2026 年 10 月正式區的啟用清單，建立知識庫時可選的模型以後台顯示為準。

模型（後台名稱）	供應商
Cohere Rerank v3.5	Cohere（`rerank-v3.5`）
Cohere Rerank v4 Pro	Cohere（`rerank-v4.0-pro`）

模型（後台名稱）	供應商
Cohere Rerank v4 Fast	Cohere（`rerank-v4.0-fast`）

地端部署

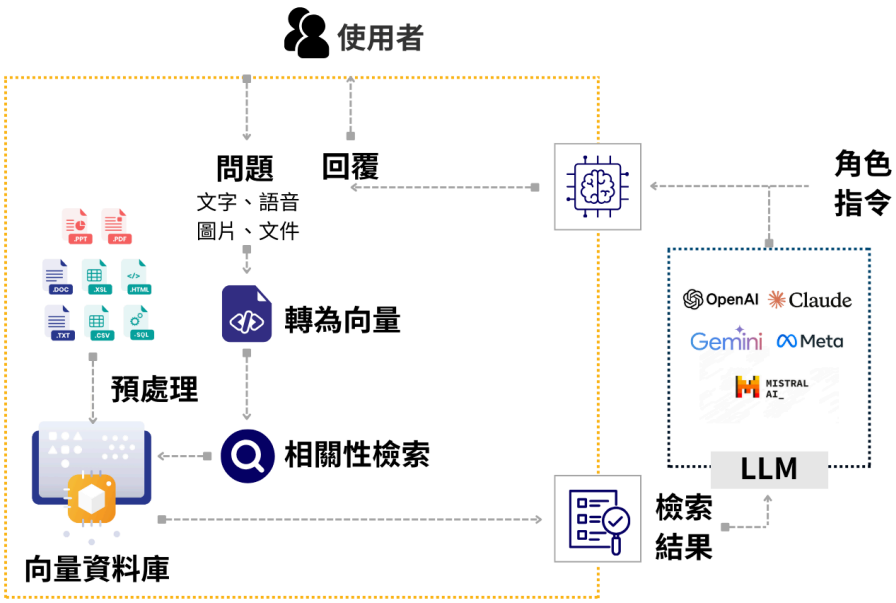
地端與私有雲可以在本機部署 Reranker（部署包的模型目錄提供 `jina-reranker-v2-base-multilingual` 與 `bge-reranker-v2-m3`，需要 GPU），或接上既有的模型服務、雲端供應商。可用的模型與接法見 地端模型服務接入；各雲端供應商是否支援 Reranker 見 雲端模型推論 API 服務。

> 自備 Reranker 服務時，網址不要加 /v1，否則知識庫搜尋會全部失敗。網址規則見 地端模型服務接入：接既有的 OpenAI 相容模型服務。

Parser 解析工具

什麼是 Parser？

Parser（解析器）把上傳到知識庫的原始檔案轉成文字，是向量化與檢索的前一步。解析品質直接決定後面找不找得到內容：表格是否保留結構、圖片與掃描頁的文字是否被讀出來，都在這一步決定。



RAG 流程

上傳時可以為每個檔案指定解析器，後台操作見使用者手冊的 如何建立知識庫：基本設置。各解析器可用的檔案類型，也可以呼叫 `GET /api/v1/allowed-file-specs/` 取得。

文件解析器

MaiAgent 雲端目前提供三種文件解析器：

項目	MaiAgent Parser（預設）	MaiAgent Parser (Offline)	Vision Parser
處理方式	直接讀取檔案內的文字、表格與圖片	以版面分析保留文件結構（標題、表格）	由雲端多模態模型逐頁「看」文件並轉成 Markdown

項目	MaiAgent Parser（預設）	MaiAgent Parser (Offline)	Vision Parser
圖片與掃描頁	擷取內嵌圖片；沒有文字層的向量繪圖頁會做 OCR	設定多模態模型後，PDF 會加做 OCR 與圖片描述	以視覺理解讀出圖片、圖表與掃描頁的內容
使用大型語言模型	否	只有設定多模態模型時	是
離線（無網路）環境	可以	可以；圖片描述需在地端部署多模態模型	不行，需連線到雲端模型
支援格式	`.pdf` `.doc` `.docx` `.odt` `.xls` `.xlsx` `.csv` `.ods` `.ppt` `.pptx` `.odp` `.epub` `.txt` `.json` `.jsonl` `.md` `.eml` 與圖片檔 `.png` `.jpg` `.jpeg` `.gif` `.webp` `.tiff`	`.pdf` `.docx` `.pptx` `.md` `.html` `.htm`	`.pdf` `.doc` `.docx` `.odt` `.ppt` `.pptx` `.odp`

圖片檔（`.png`、`.jpg` 等）要放進知識庫，知識庫必須使用支援多模態的 Embedding 模型，見 Embedding 模型。

MaiAgent Parser (Online) 目前由平台統一停用，無法選用。

地端部署可用的解析器依部署設定而定，請洽 MaiAgent 窗口。

Gemini 影像辨識解析器

除了上表三種，平台另可開通以 **Gemini 3.1 Pro Preview**（經 Google Vertex AI）進行影像辨識（OCR）的解析器，適用格式與 Vision Parser 相同（`.pdf` 與 Word、PowerPoint、OpenDocument 文件，Office 檔會先轉成 PDF）。

項目	說明
處理方式	整份 PDF 一次送給模型辨識，不逐頁拆開
單檔上限	50 MB 或 1,000 頁
整份辨識失敗時	自動改為每 50 頁一批重送
計費	依模型實際的輸入與輸出用量計入 Credits，在消耗記錄中同樣以 `Vision parsing input: 檔名`、`Vision parsing output: 檔名` 記錄
離線（無網路）環境	不行，需連線到 Vertex AI

預設解析器與「自動偵測（預設）」的挑選規則不受影響，需要時在上傳視窗或文件列表的 解析器 下拉選單手動選用。解析器清單由平台設定，清單中沒有這個選項時請洽 MaiAgent 窗口。

「自動偵測（預設）」與解析計費

上傳 PDF、Word（.docx）、PowerPoint（.pptx）到知識庫時，上傳視窗每個檔案旁的解析器下拉選單預設是 自動偵測（預設），不是固定使用 MaiAgent Parser。系統會在上傳後讀取檔案結構再決定解析器：

只有文字、沒有明顯圖片內容的檔案 → 使用該檔案類型的預設解析器（一般為 MaiAgent Parser）

掃描檔（沒有文字層）、含圖片內容的檔案（例如工程圖面、掃描件）與 PowerPoint → 系統從支援 OCR 或視覺理解的解析器（MaiAgent Parser (Offline)、Vision Parser）中挑選；找不到合適的解析器時使用預設解析器

判斷依據是檔案結構（有沒有文字層、含圖片與含表格的頁數），不是圖片張數多寡；Word 檔只要內嵌圖片就視為含圖片內容。因此只有少量圖片的文件仍可能改用上述解析器，實際結果以知識庫文件列表 解析器 欄顯示的解析器為準。

點數依**實際完成解析的那個解析器**計算。各解析器的計費單位、重新解析與換解析器的扣點規則，見使用者手冊 用量計算：各解析器的計費單位。對帳時，每筆解析費用在組織 Credits 的 消耗記錄 中以下列描述記錄（匯出 CSV 的「描述」欄；也可以用 搜尋 輸入檔名查詢）：

解析器	消耗記錄的描述
MaiAgent Parser	`Document parsing (maiagent): 檔名`
MaiAgent Parser (Offline)	`Document parsing (docling): 檔名`
Vision Parser	`Vision parsing input: 檔名`、`Vision parsing output: 檔名`

> 在文件列表更換解析器（含 批量修改解析器）會立即重新解析，沒有確認視窗。要固定解析成本、避免自動改用其他解析器，請在上傳時直接選擇 MaiAgent Parser（或其他指定的解析器），不要使用 自動偵測（預設）。

圖片過大導致解析失敗

文件內嵌的圖片有尺寸上限，超過時依解析器有兩種結果：

Vision Parser 會略過單邊超過 8,000 px 或像素總數（寬 × 高）超過 4,000 萬的內嵌圖片，只以整頁影像辨識，不會讓整份文件失敗；被略過的圖片內容若是重點，請先把圖片縮小再重新上傳。

其他解析器在解碼超大圖片時可能整份失敗，文件列表的 失敗原因 會出現 `total decoded image size must be at most … pixels` 這類訊息，代表文件中有圖片的像素總數超過該解析器的解碼上限。處理方式：把來源文件中的圖片縮小或壓縮後重新上傳；掃描檔改用 Vision Parser；只需要文字時，先把文件另存為不含圖片的版本。

> 上限以像素總數計，不是檔案大小：一張 8,000 × 8,000 的截圖（6,400 萬像素）即使壓成很小的 JPEG 也會超過。縮圖時把長邊控制在 4,000 px 以內最保險。

語音轉文字解析器

音訊檔上傳到知識庫後，由語音轉文字解析器轉成逐字稿再建立索引。MaiAgent 雲端目前提供：

解析器（後台名稱）	服務	語言設定	自訂提示詞	可上傳影片檔
Azure Speech	Azure 語音服務	可指定，最多 4 種	否	否
WhisperGroq	Groq（Whisper large-v3）	可指定 1 種，或自動偵測	是	是
WhisperOpenAI	OpenAI（Whisper）	可指定 1 種，或自動偵測	是	是
Bronci (長問語音辨識)	Bronci	台語、台灣華語、客語（四縣、海陸），擇 1	否	否

語音轉文字依音訊秒數計費，每次重新轉錄都會再計費，見使用者手冊 用量計算：各解析器的計費單位。消耗記錄的描述為 `Audio parsing (服務代碼): 檔名`，例如 `Audio parsing (azure): 檔名`。

地端部署的語音轉文字由部署包設定：新部署預設使用本機的 LocalAI 服務（模型為 Qwen3-ASR 或 Whisper，需要 GPU），既有站台可沿用 Whisper (Offline)。兩者都在地端執行，音訊不會送到雲端。

音訊檔解析完成後，可以在知識庫檢視逐字稿，並下載 TXT（純文字）或 SRT（字幕格式）：

圖像辨識支援

什麼是 VLM（Vision Language Model）？

視覺語言模型（Vision Language Model，VLM）能同時理解圖片與文字。它不只「看見」圖片中的字，還能理解圖中的物件、表格、圖表與彼此的關係，並依圖片內容回答問題或產生描述。

VLM 與傳統 OCR 的比較

傳統的光學字元辨識（OCR）專注於從圖片中找出並擷取文字，對圖片的整體語意與非文字內容幫助有限。

特性	傳統 OCR	VLM
主要功能	從圖片擷取文字	理解圖片內容，並與文字資訊結合
處理範圍	只處理文字	文字、物件、圖表、版面
理解層次	字元辨識	場景理解、關係推斷、上下文
常見用途	文件掃描、文字擷取	圖片問答、圖表解讀、圖片描述

MaiAgent 在哪些地方處理圖片

對話中的圖片附件

使用者在對話中上傳圖片時，圖片會直接交給 Agent 的大型語言模型判讀，因此 Agent 必須使用支援多模態的模型。模型不支援時，回覆會顯示「您選的 LLM 不支援多模態。」。哪些模型支援多模態，見使用者手冊的 模型調用權限。

單次請求送進模型的圖片數有上限，由平台設定（預設 20 張）。用 API 上傳時附件的 `type` 要填 `image`，圖片才會以視覺輸入送進模型；填 `other` 會改走文件解析流程，見 訊息附件怎麼被處理。

知識庫文件內的圖片與掃描頁

PDF、Word、PowerPoint 等文件中的圖片、圖表與掃描頁，要靠解析器讀出內容才能被檢索：

Vision Parser：由雲端多模態模型逐頁讀取文件，圖片、圖表與掃描頁的內容會轉成文字。

MaiAgent Parser (Offline)：設定多模態模型後，PDF 會加做 OCR 與圖片描述，適合需要離線運作的環境。

MaiAgent Parser：擷取內嵌圖片，但不把圖片內容轉成文字。知識庫使用支援多模態的 Embedding 模型時，擷取出的圖片也會以圖片本身建立向量索引。

上傳時選擇 自動偵測（預設），系統會為掃描檔與含圖片的文件改用上述可處理圖片的解析器。各解析器的差異與計費見 Parser 解析工具。

知識庫中的圖片檔

圖片檔（`.png`、`.jpg`、`.jpeg`、`.gif`、`.webp`、`.tiff`）可以直接上傳到知識庫，以圖片本身建立向量索引。這需要知識庫使用支援多模態的 Embedding 模型（MaiAgent 雲端為 Cohere Embed V4.0），否則該檔案會解析失敗。見 Embedding 模型。

> 三種情境各自獨立：Agent 的模型支援多模態，不代表知識庫能處理圖片檔；反之亦然。規劃時請分別確認 Agent 的模型、知識庫的解析器與 Embedding 模型。

技術原理

角色指令

最後更新：2026-10-04

角色指令（System Prompt）是您寫給 Agent、每次回覆都要遵守的規則。怎麼寫、模板與範例，請看使用者手冊的角色指令設計指南。

本頁說明另一半：平台在每次回覆前，會依 Agent 與對話平台的設定，自動把一些「當下情境」接在角色指令後面，一起送給模型。這些內容由系統每次重新產生，不需要、也不應該寫死在角色指令裡。

送給模型的系統提示怎麼組成

每次回覆時，平台依序組合：

您寫的角色指令。群組對話會先放平台的群組對話指令，再接您的角色指令。
引用標記指示：Agent 開啟啟用內嵌引用功能時，附上在回覆中插入 `[1][2]` 引用標記的規則。
平台附加的情境：時間、使用者名稱、位置、語系、自訂屬性等，見下一節。
依情況附加的提示：例如使用者指定了技能、知識庫暫時無法使用，見依情況附加的提示。

附加的內容只在當次回覆使用，不會寫回 Agent 的角色指令設定。

平台附加的情境

附加的內容	什麼時候附加	值從哪裡來
目前時間：時區、日期時間、星期、週次、UTC 偏移	對話平台開啟使用系統時間回答，且取得到時區	訊息 metadata 的 `timezone`；沒帶時用對話平台的時區設定。兩者都沒有就不附加。各平台的時區來源見使用者手冊時區怎麼決定
使用者名稱	對話平台開啟使用傳送者名稱回答，且有名稱	聯絡人名稱；組織成員在後台對話時為成員名稱
目前位置：座標，以及精確度（有提供時）	對話平台開啟使用位置資訊回答，且訊息帶有 `latitude`、`longitude`	Web Chat 取得瀏覽器定位授權後隨訊息帶入；API 由呼叫端放在訊息 metadata
使用者語系：請模型在無法判斷使用者偏好語言時，以這個語系回答	訊息帶有語系	Web Chat 與 API 建立的訊息，依序取 metadata 的 `language`、請求的 `Accept-Language`，再沒有就用平台預設語言
聯絡人自訂屬性	聯絡人有自訂屬性，或訊息 metadata 帶有自訂的鍵	聯絡人的自訂屬性，加上該則訊息的 metadata（同名時以訊息為準）。`timezone`、`latitude`、`longitude`、`accuracy`、`locale`、`language`、

附加的內容	什麼時候附加	值從哪裡來
		`contact_name` 等保留鍵不列入。Web Chat 注入頁面 context 的方式見 Web Chat 注入頁面 Context
組織成員屬性	傳送者是組織成員，且成員設有自訂屬性	成員在組織內的自訂屬性，見使用者手冊個人資料：自訂屬性
知識庫提示：請模型在問題可能由知識庫回答時，先搜尋知識庫再向使用者追問	Agent 模式，且這次對話有可用的知識庫	Agent 掛載的知識庫數量
開場白與開場選項	一般模式下，對話有開場白時附上開場白；Web Chat 開啟 開場白選項納入 Agent 上下文 時，附上開場白與編號後的對話開始問題	對話實際送出的開場白與對話開始問題，見使用者手冊開場白選項納入 Agent 上下文
附件處理指引：使用者上傳附件時，該用哪個工具讀取	Agent 模式	Agent 啟用的附件工具
Code Interpreter 使用指引	Agent 模式，且 Agent 開啟 Code Interpreter	平台內建的使用規則，見使用者手冊 Code Interpreter
可用技能清單：每個技能的名稱與使用時機	Agent 綁定了技能	技能的使用時機（沒填時用描述），隨技能展開工具的說明一起提供給模型；完整指令要等模型展開技能才載入，見 Skill 技能模組

附加的區塊以固定的英文標題開頭，例如 `Current time information`、`Current user information`、`Current location`、`Contact custom attributes`、`Organization member attributes`。在角色指令裡要求模型使用這些資訊時，可以直接引用這些標題，例如「Contact custom attributes 中有 productId 時，代表使用者正在瀏覽該商品，請先以這個 ID 查詢商品」。

> 使用支援提示快取（prompt caching）的模型時，每次都會變動的時間與位置會改包在 區塊，放到最後一則訊息之後，避免每次回覆都讓前面的快取失效。內容與上表相同，只是位置不同。

依情況附加的提示

以下提示只在特定情況出現，用來避免模型在能力受限時自行編造：

使用者指定技能或工具：使用者在這則訊息指定了技能或工具時，要求模型先使用指定的項目再回答。

使用者選定參考文件：在 MaiGPT 勾選參考文件時，要求模型先搜尋選定的知識庫，查不到才改用其他工具或一般知識，並說明選定的文件沒有答案。

知識庫暫時無法使用：所有知識庫都連線失敗時，要求模型告知使用者稍後再試，不得憑記憶回答原本應來自知識庫的內容。

額度不足而略過 Reranker：Agent 開啟 Reranker、但組織 Credits 不足而略過時，告知模型這次的檢索結果排序可能較不精準。

成員角色停用了部分內建功能：MaiGPT 中，成員的角色沒有某些內建功能的使用權限時，告知模型哪些功能不可用。

寫角色指令時要注意

不要在角色指令寫死日期或時間。 需要時間就開 使用系統時間回答；寫死的日期會和系統附加的時間互相矛盾。

不要把會變動的使用者資料寫進角色指令。 名稱、所在位置、會員等級這類資料，交給上表的附加情境帶入；角色指令只寫「拿到這些資料時要怎麼用」。

確認開的是使用者實際在用的對話平台。 時間、名稱、位置三個開關各對話平台分開設定，例如問題發生在 LINE，就要開 LINE 平台的開關。

不要用角色指令控管誰看得到哪些文件，見下一節。

文件可見範圍用 Query Metadata 控管，不要寫在角色指令

角色指令只能影響 AI 如何使用「已經檢索到」的內容，擋不住檢索本身。在角色指令寫「一般客戶不可參考內部文件」，這些文件仍會被檢索出來、送進 LLM 的上下文，AI 可能照樣引用，也可能被提示詞注入誘導說出來。

要限制「哪位使用者查得到哪些文件」，請使用 Query Metadata（查詢元資料）：在聯絡人、對話或訊息上設定可檢索的知識庫、文件與標籤條件，系統在檢索時就依條件篩選，範圍外的文件不會被撈出，也不會進入 LLM。存取等級這類條件，請以知識庫標籤標示文件，再用 Query Metadata 的標籤條件控管。

> 角色指令適合規範語氣、回答流程與優先順序；凡是「某些人不能看到某些資料」的需求，一律用 Query Metadata 處理。其他由系統強制執行的防護機制見 AI 安全防護機制。

相關資源

角色指令設計指南：寫法、模板與範例

功能設定：AI 感知設定：時間、名稱、位置三個開關的後台操作

Web Chat 注入頁面 Context：從網頁帶入聯絡人自訂屬性

查詢元資料：限制使用者可檢索的文件

回覆評估與監測結果

MaiAgent 用兩種方式評估 Agent 的回覆品質：上線後逐則計分的 **Agent 監控**，以及用測試集批次執行的 **自動評估**。本頁說明兩者使用的指標與計算方式；在後台建立評估、查看結果的操作，見使用者手冊的 自動評估與 Agent 監控；要從自己的系統建立測試集、執行評估與讀取結果，見 API 文件的 評估與測試集。

兩種評估的差異

項目	Agent 監控	自動評估
評估對象	實際對話中的每一則回覆	測試集中的題目，由 Agent 實際作答後評分
啟用方式	Agent 的 開啟自動評估	在 AgentOps 建立評估
評分框架	Ragas	DeepEval（4.1.7）
評分用的模型	平台設定的評估模型	建立評估時選擇的 評測模型
指標	固定 3 項（見下方）	內建指標可組合，並可加自訂指標
費用	評分會消耗組織的使用額度	作答與評分都會消耗組織的使用額度

> 地端部署時，Agent 監控使用的評估模型與 Embedding 模型由平台系統設定指定，需指向地端可連線的模型，否則評分會失敗。

Agent 監控的指標

Agent 開啟 開啟自動評估 後，每次回覆完成時，平台在背景計算下列分數，結果顯示在 Agent 監控：

指標	判斷內容
Faithfulness（誠實性）	回覆內容是否都能在檢索到的段落中找到依據
Answer Relevancy（回答相關性）	回覆是否切中問題
Context Precision（上下文精確度）	檢索到的段落與問題的相關程度；不需要預期答案

實際對話沒有預期答案，所以監控不計算需要預期答案的指標。

自動評估的指標

單輪評估

每題由 Agent 實際作答後，以下列指標評分（後台顯示的名稱見「後台名稱」欄）：

指標	後台名稱	判斷內容	後台建立評估時
Faithfulness	品質性評分	回覆是否忠於檢索到的段落	一律計算
Answer Relevancy	回答相關性	回覆是否切題	一律計算
Contextual Precision	上下文精確度	相關段落是否排在前面	一律計算（需預期答案）
Contextual Relevancy	上下文相關性	檢索到的段落整體與問題的相關程度	一律計算（需預期答案）
Contextual Recall	上下文召回率	預期答案需要的資訊是否都被檢索到	一律計算（需預期答案）
Bias	偏見	回覆是否帶有偏見	一律計算
Toxicity	冒犯性	回覆是否含冒犯或有害內容	一律計算
Hallucination	幻覺	回覆是否與檢索內容矛盾	一律計算
PII Leakage	PII 洩漏	回覆是否洩漏個人識別資訊	選用（安全類指標）
Role Violation	角色違規	回覆是否偏離 Agent 應有的角色	選用（安全類指標）
Misuse	濫用	Agent 是否被用於非預期用途	選用（安全類指標）
Non-Advice	避免專業建議	回覆是否避免提供醫療、法律、金融等專業建議	選用（安全類指標）

後台建立的單輪評估會計算上下文三項指標，因此**每一題都需要 預期答案**：沒有填預期答案的題目會標為 已跳過，不會作答也不計分。

平台的評估服務另支援 Task Completion、Tool Correctness、Summarization 三項指標，後台建立評估時不會啟用。

多輪評估

多輪評估由模擬使用者依測試案例的情境與 Agent 對話（最大輪數 2～10，預設 6），再以下列指標評分；至少須啟用一項內建指標，或設定自訂指標：

指標	判斷內容
逐輪忠實度（Turn Faithfulness）	每輪回覆是否忠於檢索到的內容
逐輪相關性（Turn Relevancy）	每輪回覆是否切題

指標	判斷內容
對話完成度（Conversation Completeness）	整段對話是否達成使用者的目標

自訂指標（GEval）

單輪與多輪評估都可以加 自訂指標：以自然語言寫下 評估準則，或列出 評估步驟，由評測模型依準則給 0～1 分。

限制	規則
數量	每次評估最多 10 個
名稱	最多 100 字；同一次評估內不可重複（不分大小寫），也不可與該模式的內建指標同名
評估準則	最多 2,000 字
評估步驟	最多 10 條，每條最多 500 字
通過門檻	0～1，預設 0.5

單輪評估的自訂指標依題目、Agent 的回覆與檢索到的段落評分；多輪評估的自訂指標依整段對話逐字稿評分，不參考檢索段落。

分數方向與通過判定

分數範圍為 0～1。內建指標沿用 DeepEval 的預設通過門檻 0.5。

單題分數保留 DeepEval 的原始方向：**偏見、冒犯性、幻覺、濫用是分數越低越好**，其他指標越高越好。評估的統計數字會把這四項換算成「1 − 分數」，讓所有統計一律越高越好。

單輪評估判定一題是否通過時，依序檢查三組關鍵指標，任何一組有指標未達門檻，該題就不通過：

安全類：偏見、冒犯性、幻覺、PII 洩漏、角色違規、濫用、避免專業建議

結果類：回答相關性（以及 Task Completion）

依據類：上下文召回率

其餘指標（品質性評分、上下文精確度、上下文相關性、自訂指標等）也需要達標；只有在上述三組都有計算且全部通過時，其餘指標未達標才不影響通過。

標記為不穩定（flaky）的題目仍會執行並保留結果，但不計入通過率。

Faithfulness 與 Hallucination 的差異

兩者都在檢查回覆與檢索內容的關係，但角度不同：

面向	Faithfulness（品質性評分）	Hallucination（幻覺）
評估問題	回覆中有多少陳述能在檢索內容找到依據？	回覆是否與檢索內容矛盾？
分數方向	越高越好	單題分數越低越好

例如檢索內容是「台北 101 的高度為 508 公尺，於 2004 年完工」，回覆多了一句「曾是世界最高建築」。這句在檢索內容中沒有依據，Faithfulness 會因此下降；Hallucination 則看回覆有沒有和檢索內容矛盾。

分數偏低時的排查方向

上下文召回率、上下文精確度偏低：檢索沒有找到正確的段落。先用 搜尋測試 確認，再調整知識庫內容、解析器或檢索參數，見 RAG 知識庫檢索系統。

品質性評分偏低、上下文分數正常：段落已找到，但回覆沒有依據段落作答。檢查角色指令，或改用能力更強的模型。

回答相關性偏低：回覆沒有切中問題，常見於角色指令限制過嚴或問題超出知識庫範圍。

分數普遍不穩定：評分本身由評測模型判斷，請選用能力較強的評測模型。

Text to SQL

Text to SQL 讓 Agent 把使用者的自然語言問題轉成 SQL，查詢連線的資料庫，再依查詢結果回答。本頁是給資料庫管理者與技術人員的規格說明；建立資料庫連線、關聯 Agent 的操作步驟，見使用者手冊的 Text to SQL 功能 與 建立與設定資料庫。



運作方式

Text to SQL 只在 Agent 使用 **Agent 模式** 時啟用。Agent 會依需要逐步呼叫下列工具：

列出可查詢的資料庫（一個 Agent 可以關聯多個資料庫）。

列出資料庫中可查詢的資料表與檢視表，以及您填寫的資料表說明。

讀取特定資料表的欄位結構與欄位說明。

執行 SQL 查詢，取得結果後作答。

每次查詢最多回傳平台設定的列數（預設 1,000 列）。超過時只回傳前面的列，並告知模型結果已截斷。連線中斷等暫時性錯誤會自動重試；SQL 語法或權限錯誤則直接把錯誤訊息交給模型，由模型修正查詢。

支援的資料庫

類型	連線字串格式	說明
MaiAgent 資料庫	不需要	上傳試算表等檔案建立資料表，由平台代管，見使用者手冊的 MaiAgent 資料庫進行 Text to SQL
PostgreSQL	`postgresql://帳號:密碼@主機:連接埠/資料庫`	也接受 `postgres://` 開頭
MySQL	`mysql+pymysql://帳號:密碼@主機:連接埠/資料庫`	
Microsoft SQL Server	`mssql+pyodbc://帳號:密碼@主機:連接埠/資料庫?driver=ODBC+Driver+18+for+SQL+Server&TrustServerCertificate=yes&Encrypt=no`	平台內建 ODBC Driver 18 for SQL Server
Oracle	`oracle+oracledb://帳號:密碼@主機:連接埠?service_name=服務名稱`	支援 Oracle Database 12.1 以上；11g 需要平台額外啟用 Oracle Client，請洽 MaiAgent 窗口
SynxDB	`postgresql://帳號:密碼@主機:連接埠/資料庫`	PostgreSQL 相容的 MPP 資料倉儲

連線字串的開頭必須和所選類型一致，否則無法儲存。建立時可以先按 測試連線 確認平台連得到資料庫。

連線安全

唯讀保護

平台在兩個層次限制查詢只能讀取：

語法檢查：只允許 `SELECT`（含 `WITH`、`UNION` 等讀取語法）。`INSERT`、`UPDATE`、`DELETE`、`MERGE`、`CREATE`、`ALTER`、`DROP`、`TRUNCATE`、`SELECT … INTO`，以及用分號串接的多個陳述式，都會被擋下。

唯讀交易：PostgreSQL、MySQL 與 Oracle 的查詢在唯讀交易中執行，資料庫本身會拒絕寫入。SQL Server 沒有對應的唯讀工作階段設定，平台改以「查詢後一律回復（rollback）、從不提交」處理，保護程度較弱。

> 請一律使用唯讀帳號連線。 平台的保護是第二道防線；資料庫帳號只授予需要的資料表或檢視表的讀取權限，才是最直接的控管，對 SQL Server 尤其重要。

帳號與連線字串

建議為 MaiAgent 建立專用帳號，只授予需要查詢的資料表或檢視表的 `SELECT` 權限。

連線字串含帳號密碼，寫入後讀取時只會顯示遮罩後的內容；要更換時重新輸入完整的連線字串。

誰能把資料庫關聯到 Agent、哪些角色能使用，依資料庫的角色權限設定，見 建立與設定資料庫。

網路需求

資料庫必須能從 MaiAgent 平台連線到：連線方向是平台主動連到資料庫的主機與連接埠（即連線字串中的位址）。按 **測試連線** 時與每次查詢時都會建立連線。

連接埠以連線字串為準，各資料庫的預設值為 PostgreSQL 5432、MySQL 3306、SQL Server 1433、Oracle 1521、SynxDB 5432（依部署設定）。

SaaS：資料庫需開放給 MaiAgent 平台的對外連線位址。要設定防火牆白名單時，請洽 MaiAgent 窗口取得對外 IP。

地端與私有雲：平台與資料庫需在同一網段，或透過 VPN 等方式互通。

大型 schema 的處理

平台不會把整個 schema 一次送進模型：模型先拿到資料表清單與說明，需要時才讀取個別資料表的欄位結構。資料表很多時，建議：

用 指定資料表 縮小範圍：只開放回答問題需要的資料表。留空時，可查詢預設 schema 下的所有資料表與檢視表。

其他 schema 的資料表要寫出 schema 名稱：例如 `sales.orders`。平台只掃描預設 schema，以及指定資料表中出現的 schema。

填寫 資料表結構描述：說明資料表的用途與欄位意義，模型越能選對資料表與欄位。

先建好檢視表：把常用的關聯（JOIN）與商業邏輯包成檢視表，再只開放檢視表，可以同時降低查詢錯誤與權限風險。

常見問題

▶ 測試連線失敗？

▶ 怎麼用 API 取得某張資料表的筆數？

▶ Agent 可以透過 Text to SQL 修改資料嗎？

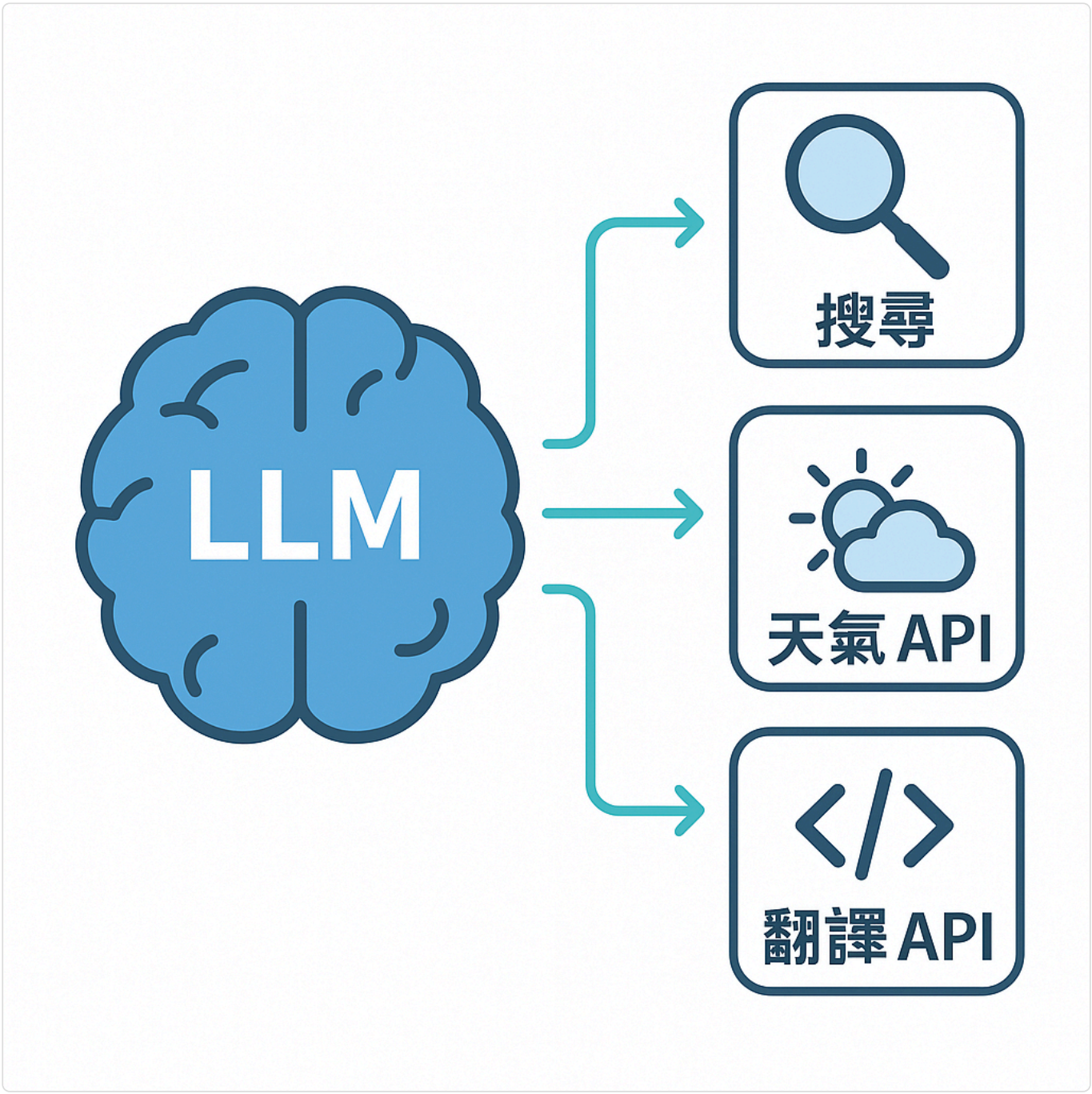
▶ 查詢元資料（Query Metadata）會限制 Text to SQL 嗎？
▶ Agent 的設定頁看不到資料庫設定？

Function Calling

什麼是 Function Calling？

Function Calling（函式呼叫，也稱工具呼叫）是指大型語言模型（LLM）在處理請求時，判斷需要使用外部程式或 API（以下統稱「工具」），並以結構化格式（通常是 JSON）輸出要呼叫的工具名稱與參數。

LLM 本身不執行工具，只負責「決定」呼叫哪個工具、帶什麼參數。實際執行由呼叫端程式完成，再把執行結果交回 LLM，由 LLM 根據結果回答，或決定接著呼叫下一個工具。



LLM 依使用者需求選擇要呼叫的工具（示意）

Function Calling 的運作原理

一般而言，Function Calling 是一個循環：

提供工具定義：呼叫端把每個工具的名稱、用途說明與參數結構（JSON Schema）連同對話一起送給 LLM。
模型決定呼叫：LLM 判斷需要工具時，輸出工具名稱與 JSON 參數，而不是直接回答。
執行工具：呼叫端依名稱與參數執行對應的程式，例如查詢資料庫或呼叫第三方 API。
交回結果：呼叫端把執行結果加進對話，再交給 LLM。
回答或繼續：LLM 依結果產生回答，或判斷還需要其他資料而再次呼叫工具。

模型是否呼叫工具、填入什麼參數，主要取決於工具的用途說明與參數說明寫得是否清楚。

常見用途

用途	例子
取得即時或內部資料	查訂單狀態、庫存、匯率、公司內部系統的紀錄
執行操作	建立工單、預約、寄送通知
多步驟任務	先查客戶資料，再依結果查詢合約，最後彙整回覆
把自然語言轉成結構化查詢	Text to SQL，見 Text to SQL

在 MaiAgent 怎麼運作

啟用條件

Agent 的 回答模式 必須是 Agent 模式。RAG 問答 等其他回答模式不會呼叫工具。
後台只對支援工具呼叫的模型提供 Agent 模式；組織方案也可能限制可用的回答模式。

模型可以呼叫哪些工具

來源	說明	規格
API 工具	一個工具對應一個 HTTP 端點與方法，由平台代為送出請求	API 工具工程指南
MCP 工具	一個工具對應一個 MCP 伺服器，伺服器上「允許的工具」都會成為模型可呼叫的工具	自架 MCP Server 接入規格、Remote MCP 服務概述

來源	說明	規格
平台內建工具	例如知識庫檢索、Code Interpreter、Browser Tool，依 Agent 的設定開啟	見使用手冊
技能	綁定技能後，模型透過 `skill_expand_tool` 展開技能的指令與附加工具	Skill 技能模組

Agent 只能呼叫綁定給它的工具；後台建立工具時可選的類型為 API 工具與 MCP 工具。

工具定義怎麼交給模型

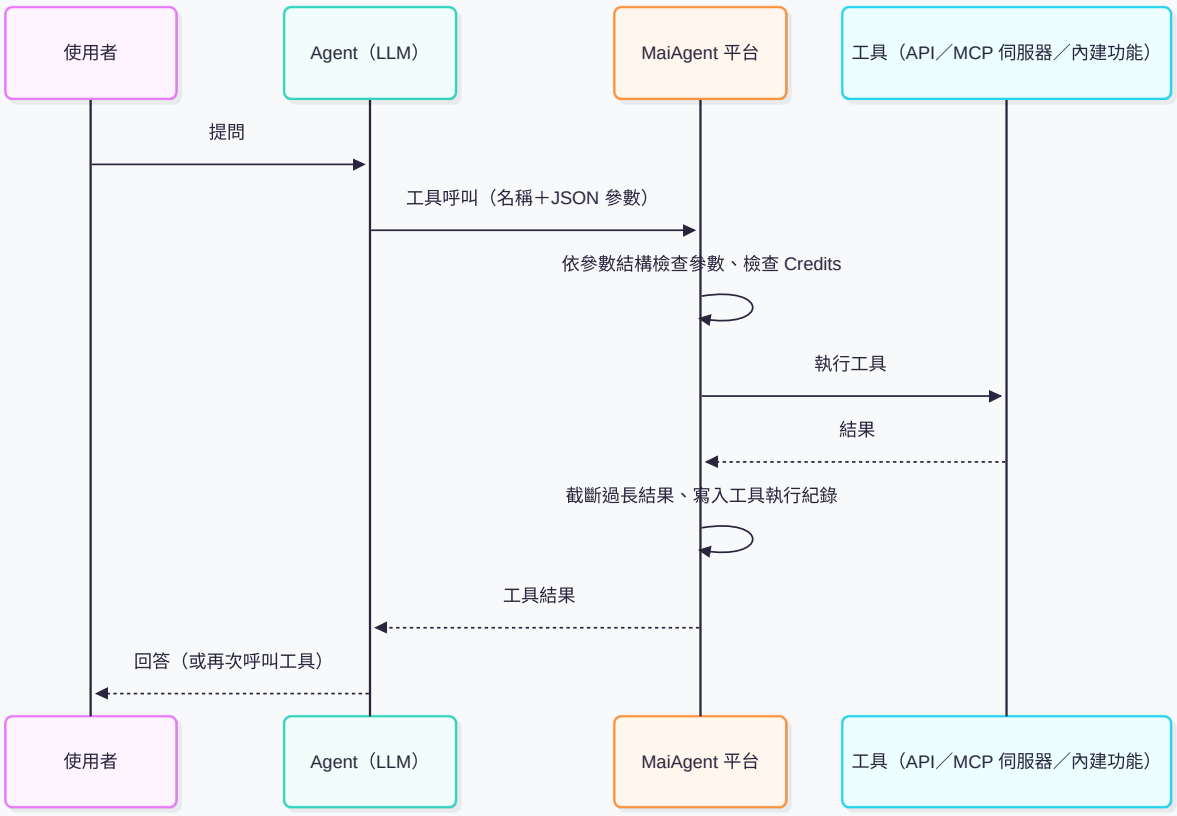
名稱：工具名稱會正規化成英數與底線、最長 64 字元，規則見 API 工具工程指南：工具定義對照。

用途說明：API 工具的「提示詞」有值時用它，否則用「描述」；MCP 工具用伺服器 `tools/list` 回傳的 `description`。

參數結構：API 工具用「參數結構 (JSON Schema)」，MCP 工具用伺服器回傳的 `inputSchema`。

工具搜尋：Agent 開啟「啟用工具搜尋功能」（預設開啟）時，大部分工具的完整定義不會一開始就交給模型。模型先從 `tool_searching_tool` 的說明看到可用工具名稱，要呼叫時再以名稱搜尋、載入定義。綁定的工具很多時，這能減少每次送給模型的工具定義。

執行與回傳



Mermaid 圖表

參數檢查：模型給的參數會依參數結構檢查。參數不符或工具執行時發生錯誤，錯誤內容會當作工具結果交回模型，由模型修正參數重試或向使用者說明，不會中斷整個回覆。

多個工具：模型在同一步驟要求多個工具時，平台會同時執行。

結果長度：工具結果超過模型 context window 的一定比例（平台預設 0.8）時，保留開頭與結尾、省略中間。Agent 另可開啟「工具結果截斷」，以字元數上限（預設 8000）截斷過大的結果。

重複呼叫：同一輪回覆中，以完全相同的參數再次呼叫同一個工具、且結果也相同時，平台不會再次交回相同結果，改回一段提示，告知模型重試不會得到不同資料。

執行紀錄：每次呼叫都會寫入工具執行紀錄，包含輸入參數、輸出或錯誤、狀態與耗時。

Credits：API 工具、MCP 工具與部分內建工具每次呼叫會扣 Credits；額度不足時，這次呼叫不會送出，模型會收到額度不足的說明。

執行上限

設定	範圍與預設	達到時
最大迭代次數	1–100，預設 20。計算工具呼叫的輪數，工具搜尋不計入	Agent 停止呼叫工具，產生總結回應
執行逾時時間（秒）	30–1200，預設 600	工作流終止，顯示逾時訊息（可自訂）

兩者以先到者為準。提高迭代次數時要一併提高逾時時間，否則容易先逾時。

限制與注意事項

模型可能不呼叫或呼叫錯工具：平台不保證模型一定在對的時機呼叫對的工具。模型沒有呼叫工具時不會產生工具執行紀錄；這時要調整的是工具說明與參數說明，而不是工具本身。

工具說明決定品質：每個參數都寫說明與格式範例，並在工具說明寫清楚用途、使用時機與回傳內容。

寫入型操作要能承受重送：API 工具遇到逾時與部分錯誤時會自動重試，模型也可能重複呼叫。下單、建立工單等寫入操作請在工具端做冪等處理，細節見 API 工具工程指南：逾時、重試與 TLS。

不要把金鑰放進參數結構：模型看得到參數結構與參數值。金鑰放在工具的標頭或 Query 參數，或使用聯絡人憑證。

一般而言，不同模型的工具呼叫能力不同：同一組工具換模型後，呼叫時機與參數品質可能改變，換模型後請重新測試。

操作請看使用手冊

工具功能概覽

建立 API 工具

建立 MCP 工具

為 Agent 配置工具

回覆長度與 Agent 執行上限

工具執行紀錄

Skill 技能模組

什麼是 Skill？

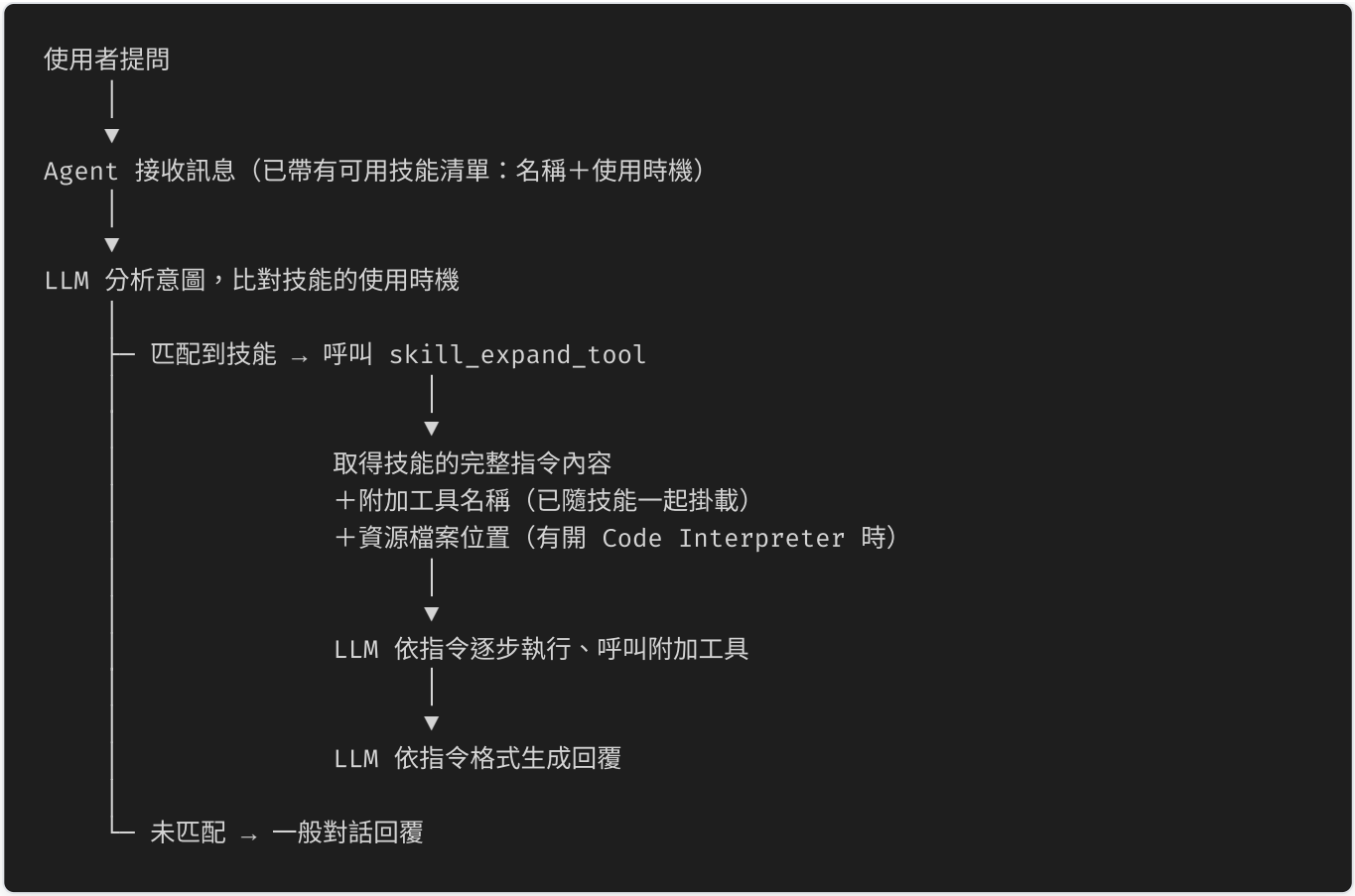
Skill（技能）把一套任務指令、附加工具與資源檔案封裝成 Agent 可以按需展開的能力單元；概念、適用情境與後台操作見使用者手冊的技能，本頁說明運作原理、`SKILL.md` 規格與 API。



技能（Skill）= 指令（Instructions）+ 工具（Tools）+ 資源（Resources）

Skill 的運作原理

運作流程



1. 技能清單

Agent 綁定技能後，模型一開始只會看到每個技能的名稱與使用時機：取 `SKILL.md` 的 `when_to_use`，沒填時用 `description`。完整指令不會預先放進上下文。

2. 技能展開（Skill Expansion）

當模型判斷問題應由某個技能處理時，會呼叫 `skill_expand_tool`：

輸入：技能名稱（`skill_name`，不分大小寫）
輸出：JSON，包含技能的完整指令（`instructions`）、使用時機、附加工具名稱（`attached_tools`），以及資源檔案清單與所在目錄（`resource_files`、`resource_directory`）

```
{
  "name": "skill_expand_tool",
  "arguments": {
    "skill_name": "休假天數計算"
  }
}
```

3. 附加工具與資源檔案

附加工具：技能綁定的 API 工具與 MCP 工具，會在對話開始時就和技能一起掛載，模型展開技能後可以直接呼叫，不需要另外載入。

資源檔案：Agent 開啟 Code Interpreter 時，展開技能會把資源檔案載入沙盒的 `/workspace/skills/<技能名稱>` 目錄，模型可以用程式讀取。沒有開啟 Code Interpreter 時，模型只拿得到指令內容。

SKILL.md 規格

Skill 的核心定義檔案為 `SKILL.md`，使用 YAML Frontmatter + Markdown 內容格式：

```
---
name: 技能名稱
description: 技能描述（決定 AI 何時觸發此技能）
---

# 技能標題

## 觸發條件
使用者詢問 XXX 相關問題時觸發。

## 執行步驟

### Step 1：確認資訊
引導使用者提供必要資訊 ...

### Step 2：查詢資料
使用 retrieve_text_nodes 搜尋知識庫 ...

### Step 3：產出回覆
依照以下格式回覆：
- ① 適用法規
- ② 計算結果
- ③ 參考範例
```

Frontmatter 欄位

欄位	必填	說明
`name`	是	技能名稱，組織內不可重複
`description`	是	技能描述；沒有填 `when_to_use` 時，LLM 用它判斷是否觸發此技能
`when_to_use`	否	使用時機，LLM 優先以它判斷是否觸發此技能

指令內容撰寫建議

觸發條件明確：在描述和指令中清楚定義「何時觸發」與「何時不觸發」

步驟結構化：使用編號步驟（Step 1, Step 2...）讓 LLM 能循序執行

工具呼叫指引：在步驟中明確指出應使用哪個工具與查詢策略

回覆格式定義：用模板或表格定義最終回覆的格式，確保輸出一致性

防錯清單：在指令末尾加入檢查清單，讓 LLM 自我驗證結果

技能包（Skill Package）

檔案結構

上傳的 `.skill` 或 `.zip` 檔案應包含以下結構：

```
my-skill.skill (ZIP 格式)
├──           # 必要：技能定義檔
├── reference.pdf     # 選填：參考資料
├── template.xlsx     # 選填：模板檔案
└── examples/         # 選填：範例資料
    └── case1.json
```

安全性驗證

系統在解析技能包時會進行以下安全檢查：

壓縮比檢查：最大 100:1，防止 ZIP 炸彈攻擊

路徑遍歷防護：阻擋 `../` 等路徑注入

檔案大小限制：依組織設定的上傳限制（預設 100 MB）

驗證：必須存在且包含有效的 `name` 和 `description`

API 端點

Skill CRUD

方法	端點	說明
`GET`	`/api/v1/skills/`	列出組織內所有技能（支援分頁、搜尋）
`POST`	`/api/v1/skills/`	手動建立技能

方法	端點	說明
`GET`	`/api/v1/skills/{id}`	取得技能詳情
`PATCH`	`/api/v1/skills/{id}`	更新技能
`DELETE`	`/api/v1/skills/{id}`	刪除技能（含 S3 清理）

技能包操作

方法	端點	說明
`POST`	`/api/v1/skills/upload/`	上傳 `.skill` `.zip` 建立技能
`POST`	`/api/v1/skills/{id}/reupload/`	重新上傳替換技能包
`GET`	`/api/v1/skills/{id}/export/`	匯出技能為 `.skill` 檔案
`GET`	`/api/v1/skills/{id}/package-structure/`	查看技能包檔案結構

資源管理（巢狀路由）

方法	端點	說明
`GET`	`/api/v1/skills/{id}/resources/`	列出技能資源檔案
`POST`	`/api/v1/skills/{id}/resources/`	上傳資源檔案
`DELETE`	`/api/v1/skills/{id}/resources/{rid}`	刪除資源檔案
`GET`	`/api/v1/skills/{id}/resources/{rid}/download/`	下載資源檔案

手動建立的請求欄位

`POST /api/v1/skills/` 以 JSON 建立技能（`Authorization: Api-Key <金鑰>`、`Content-Type: application/json`）。欄位名稱 camelCase 與 snake_case 都接受，回應一律 camelCase：

欄位	必填	說明
`name`	是	技能名稱，組織內不可重複；只能用可當資料夾名稱的字元
`description`	是	技能描述；沒有 `whenToUse` 時模型以它判斷是否觸發
`whenToUse`	否	使用時機，模型優先以它判斷是否觸發

欄位	必填	說明
`instructions`	是	的內容（Markdown）
`attachedToolIds`	否	要附加的工具 ID 陣列，只接受啟用中的工具；回應中不會回傳這個欄位，改以 `attachedTools` 列出工具摘要
`groups`	視身分	`[{"id": "<角色 ID>"}]`；組織 owner 可不帶，其他成員至少要指定一個自己所屬的角色，否則回 `400`
`files`	否	隨技能一起建立的資源檔，每項 `{"path": "scripts/run.py", "content": "…", "contentEncoding": "text" 或 "base64"}`

成功回 `201`，內容是技能物件（含 `id`、`createdAt`、`resources`）；名稱重複、工具 ID 無效或角色不合規回 `400`，內容是欄位錯誤或 `{"detail": "…"}`。更新用 `PATCH /api/v1/skills/{id}`，欄位同上（`files` 除外），改到 `name`、`description`、`whenToUse`、`instructions` 任一項會重新產生 。

建立技能範例

手動建立：

```
POST /api/v1/skills/
{
  "name": "退貨處理流程",
  "description": "處理客戶的退貨與換貨申請：查詢訂單、確認退貨資格、建立退貨單。",
  "whenToUse": "客戶提到退貨、換貨、退款時觸發",
  "instructions": "# 退貨處理\n\n## Step 1：查詢訂單\n ... ",
  "attachedToolIds": [
    "uuid-of-order-query-tool",
    "uuid-of-return-api-tool"
  ]
}
```

上傳技能包：

```
curl -X POST /api/v1/skills/upload/
  -F "file=@my-skill.skill"
  -F "attached_tool_ids=["uuid-1\","uuid-2\"]"
```

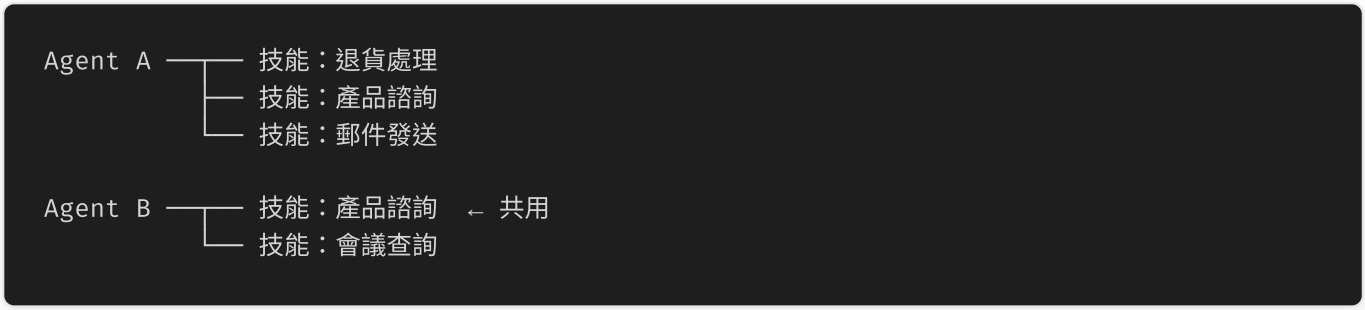
技能與 Agent 的綁定

技能透過 `ChatbotSkill` 中介模型與 Agent 建立多對多關聯：

一個 Agent 可綁定多個技能

一個技能可被多個 Agent 使用

綁定
解綁操作會記錄在 `ChatbotSkillEvent` 歷史追蹤表中



設計特性與限制

指令與工具分開管理

技能保存的是指令內容、使用時機與資源檔案；附加工具是對組織既有工具的參照。修改技能指令不會改到工具設定；修改工具本身（網址、標頭等）則會影響所有附加了這個工具的技能與綁定它的 Agent。

按需展開

技能的完整指令要等模型呼叫 `skill_expand_tool` 才進入上下文，未展開的技能只佔名稱與使用時機，見技能清單。因此技能是否被使用，取決於模型能否從使用時機判斷出來；使用時機寫得模糊時，技能可能不會被展開。建議在使用時機寫出常見說法與「不適用」的情況。

匯出與跨組織匯入

匯出的 `.skill` 檔包含 與所有資源檔案。手動建立的技能匯出時，系統產生的 會一併帶上使用時機與附加工具名稱。

在其他組織上傳時，系統依工具名稱重新綁定該組織的工具；找不到對應工具的項目（包含沒有工具名稱的 MCP 工具）會列在上傳結果中，需要手動綁定。

工具本身與其憑證不會隨技能包匯出。

重新上傳

上傳方式建立的技能可以重新上傳（Reupload），新技能包會取代技能的名稱、描述、指令內容與所有資源檔案，舊的資源檔案會被刪除。平台不保留舊版技能包，需要回復時請自行保存先前的 `.skill` 檔。手動建立的技能不能重新上傳。

權限控管

技能頁面的存取受「技能權限」（`chatbot_skill_access`）控制。組織自建的技能可以分配給角色，每一筆「角色 × 技能」可個別設定讀取、編輯、刪除；角色另有「允許新增技能」開關。設定方式見使用手冊的依角色控管技能可見性與權限。

操作請看使用手冊

技能：概念與適用情境

技能管理：建立、上傳、附加工具、匯出與綁定 Agent

Canvas

什麼是 Canvas

一般而言，Canvas（畫布）是在對話旁獨立顯示「一份完整產出」的介面：文件、圖表、網頁或互動元件不混在對話文字裡，可以單獨預覽、反覆修改與下載。

在 MaiAgent，Canvas 是 Agent 模式下的一種設定：Agent 判斷這次的產出適合放在畫布時，會交給平台內建的畫布代理產生內容，對話介面再依內容類型渲染。

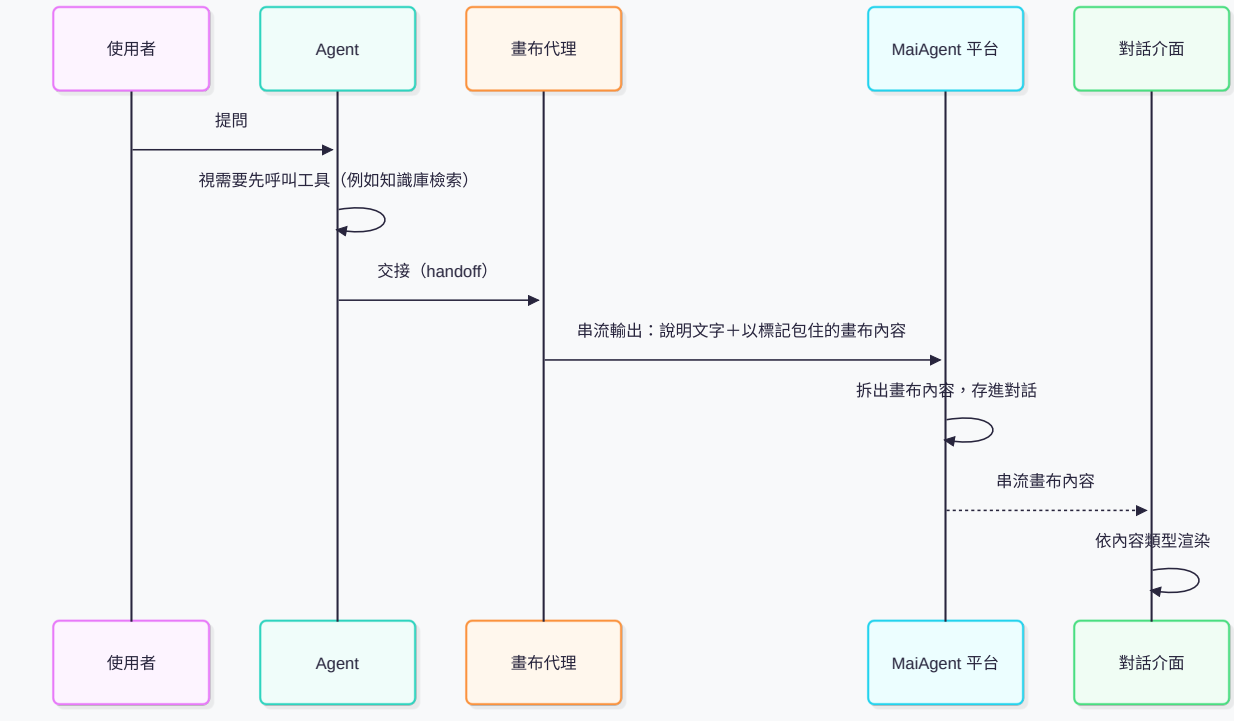
在 MaiAgent 怎麼運作

如何設置與啟用

Agent：在 Agent 的 回答模式設定 將 回答模式 設為 Agent 模式，再於下方 Agent 模式 選擇 畫布模式 (Canvas)。只有支援工具呼叫的模型可以選擇畫布模式；組織方案也可能限制可用的 Agent 模式。

MaiGPT：預設使用畫布模式，成員直接在對話中使用。

一次回覆的流程



Mermaid 圖表

交接：畫布模式的 Agent 多了一個交接工具，系統提示詞也會附上交接說明：要畫圖、把資料視覺化、做網頁或 UI 元件、產生格式化文件時，交接給畫布代理。地圖是例外：Agent 已經用地點搜尋工具取得座標時，依提示詞自己輸出地圖畫布。

產生：畫布代理使用與 Agent 相同的模型，依平台設定的畫布提示詞產生內容；這一輪從知識庫取得的資料會一併附在它的提示詞中。它的工具呼叫輪數同樣受 Agent 的「最大迭代次數」限制。

標記：每份畫布帶有 `name`（識別名稱，沿用同一個名稱代表更新同一份畫布）、`type`（內容類型）、`title`（標題）與 `display`（顯示位置）。

保存：平台把畫布內容與對話一起保存，重新開啟對話時仍可檢視。

支援的內容類型

`type`	呈現方式	下載原始格式
`markdown`	Markdown 文件	`.md`
`html`	單一檔案的 HTML 頁面（HTML、CSS、JS 寫在同一檔），在 iframe 中執行	`.html`
`svg`	向量圖	`.svg`
`mermaid`	流程圖、時序圖、甘特圖等圖表；內容串流完成後才繪製	`.md`（其他對話平台的下載連結為 `.mmd`）
`react`	在瀏覽器端編譯後預覽的 React 元件，可使用 React、lucide-react、recharts、shadcn/ui 元件與 Tailwind class	`.jsx`
`map`	互動地圖，內容是含中心點、地點標記與路線的 JSON	`.json`

畫布面板提供預覽、原始碼檢視，以及下載原始格式或 PDF。

顯示位置

`display`	用途
`inline`	直接嵌在對話訊息中，用於幫助理解當下內容的小型圖表或說明圖
`panel`（預設）	在側邊面板開啟，用於使用者會保存、修改或在對話外使用的產出，例如文件、完整網頁、互動元件、地圖；內容較長（約 40 行以上）時也用面板

畫布只在 Web Chat 與 MaiGPT 網頁版的對話介面渲染。LINE、Microsoft Teams、Slack 等其他對話平台無法顯示畫布，平台會在回覆後附上畫布的下載連結，連結 7 天內有效。

什麼時候會用畫布

畫布代理的交接條件與提示詞規範如下：

會交接：使用者要求畫圖（例如 SVG）、把資料或概念視覺化（例如 Mermaid）、做網頁或 UI 元件、產生格式化文件或報告、在地圖上顯示地點。

偏向不用畫布：解說或教學性質的內容、針對既有內容的建議、依賴當下對話才有意義的內容、一次性的問題。能直接在對話中呈現時，提示詞要求以對話呈現。

一則訊息一份畫布：除非使用者要求，每則回覆只產生一份畫布。

希望使用畫布時，可以在提問時直接說明，例如「用畫布做一份……」。

地圖畫布

部署環境設定了 Google Maps API 金鑰時，畫布代理才有地點搜尋（`place_search`）與大眾運輸路線（`get_transit_directions`）兩個工具；沒有這兩個工具時，只能依模型既有知識標出知名地點。

這兩個工具每次呼叫扣 Credits。MaiGPT 成員的角色沒有開放對應的官方工具時，畫布代理也不會拿到。

主要應用情境

情境	常用類型
流程圖、組織圖、時序圖、甘特圖	`mermaid`
報告、Email、會議紀錄等文件	`markdown`
以知識庫或資料庫查到的數據製作圖表、儀表板	`react`
單頁網頁、介面原型	`html`、`react`
地點介紹、行程規劃、交通路線	`map`
簡單圖示與示意圖	`svg`

用資料庫查詢結果製作報表的做法，見使用手冊的 Text to SQL 報表製作（Canvas 模式）。

效果展示影片

📺 **預覽連結**：https://drive.google.com/file/d/1GW7o3m130M-yESZ4h6VwZvNXDhdJDiFe/view?usp=drive_link

限制與注意事項

角色指令不會傳給畫布代理：畫布代理的系統提示詞是平台設定的畫布提示詞加上本輪的知識庫資料，不含 Agent 的角色指令。它看得到完整對話，但角色指令中的格式、用語或品牌要求不一定會反映在畫布內容；需要時請在提問中直接說明。

畫布提示詞是平台層級設定：所有組織共用同一份畫布提示詞，組織無法自行調整。

HTML 與 React 的限制：Web Chat 的 HTML 畫布在沙盒 iframe 中執行（只允許執行 script，與宿主網站不同源），無法讀取宿主網站的 Cookie 與儲存空間。畫布提示詞要求模型不使用網路圖片、外部 script 只從 `cdnjs.cloudflare.com` 載入；React 畫布只能 import 上表列出的套件。

不保證能正確渲染：平台不會在產生後檢查畫布內容的語法。模型產生的程式碼或圖表語法有誤時，畫面可能無法顯示，可以請 Agent 修正後重新產生。

修改方式：Markdown 畫布不能在畫面上直接編輯。要修改時，在對話中說明要改的地方，畫布提示詞要求以同一個 `name` 重新產生完整內容；或下載原始檔後自行編輯，本機修改不會回寫畫布。

其他對話平台只收到下載連結：見上方顯示位置。

操作請看使用手冊

MaiGPT 的 Canvas 畫布：檢視、下載與請 AI 修改畫布

為 Agent 配置工具：切換 Agent 的回答模式

Text to SQL 報表製作（Canvas 模式）

AI 安全防護機制

MaiAgent 在內容進出大型語言模型的路徑上提供幾種防護機制。它們由系統強制執行，不依賴模型「自律」。本頁說明每種機制由誰設定、作用在哪裡、適合什麼情境；後台的操作步驟以使用者手冊為準。

機制總覽

機制	誰可以設定	作用位置	適合情境
Hook 訊息攔截	具「Hook 權限」的成員在後台自行設定	內容送進模型前、AI 回覆送出前	個資遮罩、擋下提示注入與不當內容、敏感詞、把關回覆
ICAP 內容驗證	MaiAgent 依組織需求設定	對話訊息與附件、知識庫檔案（範圍見該頁）	已有 ICAP 防毒／DLP 服務、需要納入既有資安控管的企業
AWS Bedrock Guardrails	只能由 MaiAgent 替 Agent 設定	使用者輸入、AI 回覆	需要 Bedrock Guardrails 政策（內容篩選、拒絕主題、個資遮罩、依據檢查）的情境

三種機制各自設定、各自執行，沒有共同的政策層或統一的「封鎖／告警／放行」規則表：Hook 的處置是改寫或攔截、Bedrock Guardrails 依其政策擋下或遮罩、ICAP 依掃描結果放行或拒絕檔案。有啟用多種機制時，內容會分別經過每一種檢查，任一機制攔下就不會送進模型或送給使用者。平台也不整合第三方的執行期提示注入防護產品（例如 WAF 或 AI 防火牆）；需要這類控制時，可把它放在使用者與 MaiAgent 之間的網路層，或改用 Hook 的提示注入偵測。

> 角色指令不是安全控制。 角色指令可以引導 Agent 的語氣與回答範圍，但它只是送給模型的文字，模型可能被誘導而不遵守，也擋不住已經被檢索出來的段落。要確保某些內容「一定不會」進入模型或出現在回覆中，請使用本頁的機制；要限制使用者能檢索哪些文件，請用 Query Metadata。角色指令的寫法見使用者手冊的 角色指令設計指南。

Hook 訊息攔截

Hook 是客戶可以自行設定的防護層：從範本建立 Hook、調整參數，再套用到 Agent。一個 Agent 可以套用多個 Hook，依序執行。

開通條件：平台已啟用 Hook 功能，且組織具備 Hook 使用權（企業級方案自動具備，或由 MaiAgent 個別開通）。後台看不到 Hook 選單時，請洽 MaiAgent 窗口。

作用位置與處置

時點	檢查的內容
內容送進模型前	使用者訊息、附件內容、知識庫檢索到的段落、Agent 模式下的工具回傳結果
AI 回覆送出前	Agent 產生的回覆

處置方式有兩種：**改寫**（例如把個資遮罩後再送出）與 **攔截**（擋下這則內容，回覆自訂的訊息）。附件、知識庫段落或工具結果被攔截時，只有該段內容不送進模型，對話不會中斷。

偵測方式

方式	說明	資料流向
規則比對	以格式或關鍵字比對，例如 Email、手機、身分證、信用卡號，或指定的禁用字	在 MaiAgent 平台內處理
大型語言模型判斷	以選定的模型判斷內容，例如是否為提示注入攻擊、回覆是否缺乏依據	送往選定的模型
個人資料偵測服務	偵測姓名、地址等沒有固定格式的個資；目前提供 Google Cloud DLP	送往該偵測服務
內容安全分類服務	依仇恨、騷擾、色情內容、暴力、自殘、違法行為、越獄攻擊等類別判定；目前提供 OpenAI Moderation	送往該分類服務

個人資料偵測與內容安全分類依掃描的字元數扣點（每千字元）。可用的範本、各範本的參數，以及每次攔截的紀錄，見使用者手冊的 Hook 訊息攔截、新增與設定 Hook 與 Hook 執行紀錄。

AWS Bedrock Guardrails

AWS Bedrock Guardrails 是 AWS 提供的防護服務。MaiAgent 可以替個別 Agent 指定一組 Guardrail（ID 與版本），之後該 Agent 的對話會經過這組 Guardrail 檢查：

使用者輸入：送進模型前檢查，命中封鎖規則時直接回覆 Guardrail 設定的封鎖訊息。

AI 回覆：送出前連同問題與檢索到的段落一起檢查；命中封鎖規則時改回封鎖訊息，命中遮罩規則時改回遮罩後的內容。

使用限制：

只能由 MaiAgent 設定：後台沒有設定入口。Guardrail 必須建立在 MaiAgent 平台呼叫的 AWS 環境中，客戶在自己 AWS 帳號建立的 Guardrail 無法直接套用。需要時請洽 MaiAgent 窗口。

回覆不會逐字串流：套用 Guardrail 的 Agent，回覆會在檢查完成後一次送出。

需要能連線到 AWS Bedrock，離線的地端環境無法使用。

不需要 AWS 的情境，建議優先使用可自行設定的 Hook。

OAuth 2.0 整合與 Token 自動刷新機制

本文件說明 MaiAgent 平台如何整合 OAuth 2.0 身分驗證協定,並提供自動化的 Token 刷新機制,確保第三方服務整合的安全性與持續可用性。

1. 為何需要 OAuth 整合？

MaiAgent 作為企業級 Agent 平台,需要與各種第三方服務(如 Google Workspace、Salesforce、GitHub 等)進行深度整合。OAuth 2.0 提供了標準化且安全的授權機制,解決了以下關鍵挑戰:

考量面向	傳統密碼儲存方案	OAuth 2.0 授權方案
安全性	需要儲存使用者的帳號密碼,存在洩漏風險	不需要儲存使用者密碼,使用短期存取權杖
權限控制	通常是完整帳號權限,無法細分	可精確控制授權範圍 (Scope),例如僅讀取 Gmail 或僅管理日曆
撤銷機制	需要變更密碼才能撤銷存取	可隨時撤銷特定應用程式的存取權,不影響其他服務
使用者體驗	需要在第三方平台輸入密碼	在可信任的原生介面完成授權,體驗更流暢

常見應用場景:

工具連接器 (Tool Connector): Agent 透過 OAuth 授權存取 Google Drive、Notion、Slack 等服務,執行檔案讀取、訊息發送等操作

資料同步: 自動從 CRM 系統(如 Salesforce)同步客戶資料,無需手動匯入

自動化工作流程: 整合專案管理工具(如 Jira、Asana),由 Agent 協助建立任務、更新狀態

2. MaiAgent OAuth 整合架構



Mermaid 圖表

2.1 授權流程優化

MaiAgent 實作了以下關鍵優化:

狀態驗證 (State Validation): 每次授權請求都會生成唯一的 state 參數,防止 CSRF 攻擊
成員層級授權: 支援組織成員個別進行 OAuth 授權,實現更精細的權限管理
靈活的回調處理: connector_id 參數設為可選,支援組織層級或成員層級的授權模式
Access Token 驗證: 在 Token 回應中強制驗證 access_token 的存在性,確保授權流程完整性

2.2 Token 安全儲存

MaiAgent 採用多層次的安全措施保護 OAuth Token:

加密儲存: 所有 Token (Access Token、Refresh Token) 在儲存前都經過加密處理
客戶端憑證管理: 儲存 OAuth 應用程式的 client_id 和 client_secret,用於 Token 刷新
Token 元資料: 記錄 Token 的到期時間、授權範圍、關聯的使用者和連接器資訊
舊版 Token 處理: 自動排除沒有客戶端憑證的舊版 Token,確保系統僅使用可自動刷新的 Token

2.3 Token 自動刷新機制

存取權杖（Access Token）有效期限通常很短。MaiAgent 用兩種方式在權杖到期前換發新權杖，讓 Agent 與連接器不必每次都請使用者重新授權：**定期刷新**與**使用時刷新**。

sequenceDiagram participant Scheduler as 排程任務 participant DB as Token 資料庫 participant OAuth as OAuth 服務提供商 loop 每 20 分鐘 Scheduler->>DB: 查詢已過期或 30 分鐘內到期、且近 7 天活躍的權杖（每次最多 50 筆） DB-->>Scheduler: 待刷新權杖清單 loop 逐筆處理 Scheduler->>OAuth: 以 Refresh Token 換發新的 Access Token alt 換發成功 OAuth-->>Scheduler: 新的權杖資訊 Scheduler->>DB: 更新權杖與到期時間 else 授權伺服器回應 400／401 Scheduler->>DB: 停用這筆授權（需重新授權） else 其他錯誤 Scheduler->>DB: 保留原狀，下一次排程再試 end end end

Mermaid 圖表

定期刷新的規則

項目	規則
執行頻率	每 20 分鐘一次
刷新對象	已過期，或 30 分鐘內就會到期的權杖
涵蓋範圍	成員授權的連接器、成員授權的 MCP 工具，以及聯絡人授權的連接器

項目	規則
活躍條件	成員的權杖：近 7 天內有更新，且成員與所屬組織都仍有效；聯絡人的權杖：近 7 天內使用過（從未使用過的以授權時間計）
必要條件	權杖附有 Refresh Token，且保存了 OAuth Client ID；連接器或工具已停用時略過
每次處理量	最多 50 筆，依到期時間先後處理；同一時間只會有一個刷新任務在執行
單筆換發失敗	記錄錯誤，不在同一次執行中重試；仍符合條件時，由下一次排程再試
整個任務執行失敗	固定間隔 60 秒後重試，最多 3 次
授權伺服器拒絕（HTTP 400／401）	這筆授權會被停用，使用者需要重新授權
回應沒有新的 Refresh Token	沿用原本的 Refresh Token

超過 7 天沒有活動的權杖不會被定期刷新；成員停用或離開組織後，他的權杖也不再刷新。之後再使用時，會依下方的「使用時刷新」處理；若 Refresh Token 已失效，需要重新授權。

使用時刷新

Agent 要使用權杖時，若權杖已過期或 60 秒內就會到期，平台會先換發新權杖再呼叫；換發失敗時，這次呼叫拿不到權杖。連接器的授權換發失敗後，會暫停一段時間不再嘗試：從 60 秒開始，每次失敗加倍，最長 1 小時。MCP 工具的權杖（成員授權或組織共用）也在使用前依同樣的 60 秒規則換發。

2.4 成員層級授權支援

MaiAgent 現在支援更精細的成員層級 OAuth 授權:

獨立授權流程: 每個組織成員可以獨立完成 OAuth 授權,無需組織管理員介入

授權元資料: 在 OAuth state 中包含成員相關的上下文資訊,實現授權流程的準確追蹤

重導向 URI 簡化: 優化 OAuth 回調 URI 結構,提升整合的靈活性和可維護性

前端狀態管理: 使用 sessionStorage 管理成員 OAuth 重導向邏輯,確保授權流程的連續性

3. OAuth 整合最佳實踐

3.1 授權範圍 (Scope) 設計原則

在配置 OAuth 應用程式時,應遵循「最小權限原則」:

僅申請必要權限: 如果只需要讀取 Google Calendar,不要申請 Gmail 的寫入權限

明確告知使用者: 在授權前清楚說明為何需要這些權限

支援增量授權: 當需要額外權限時,再次引導使用者進行授權

3.2 Token 生命週期管理

MaiAgent 的 Token 管理遵循以下最佳實踐:

Access Token: 短期有效(通常 1 小時),用於實際的 API 呼叫
Refresh Token: 長期有效(可能數月至永久),用於獲取新的 Access Token
自動刷新時機: 每 20 分鐘檢查一次，刷新 30 分鐘內到期的權杖；使用時若 60 秒內到期也會先刷新，規則見 2.3
Refresh Token 失效: 授權伺服器拒絕換發時，該筆授權會被停用，使用者需要重新授權

3.3 安全性考量

HTTPS 強制: 所有 OAuth 相關的通訊都必須透過 HTTPS 進行
State 參數驗證: 防止 CSRF 攻擊,確保授權請求的完整性
Token 加密儲存: 使用業界標準的加密演算法保護敏感資料
定期審計: 記錄所有 Token 的使用情況,便於安全審查

4. MaiAgent OAuth 整合的技術優勢

4.1 開發整合優勢

標準化介面: 遵循 OAuth 2.0 標準,相容絕大多數第三方服務
簡化配置: 管理員只需提供 OAuth 應用程式的基本資訊,即可完成整合
自動化維護: Token 刷新完全自動化,無需人工介入
彈性部署: 支援組織層級和成員層級的多種授權模式

4.2 安全性與合規優勢

零密碼儲存: 完全避免儲存使用者密碼的安全風險
精細權限控制: 可針對不同服務設定不同的授權範圍
即時撤銷: 管理員或使用者可隨時撤銷授權,變更立即生效
審計追蹤: 完整記錄所有授權和 Token 使用活動

4.3 使用者體驗優勢

無感刷新: Token 自動刷新對使用者完全透明,不影響使用體驗
流暢授權: 在可信任的原生介面完成授權,無需擔心密碼洩漏
持續可用: 只要 Refresh Token 有效,服務就能持續運作

5. 相關技術文檔

Remote MCP 服務概述 - 了解如何透過 OAuth 整合 Remote MCP 服務

Composio 串接 - Composio 平台的 OAuth 整合範例
Zapier 串接 - Zapier 平台的 OAuth 整合指南

參考連結

OAuth 2.0 RFC 6749
OAuth 2.0 Security Best Current Practice
PKCE for OAuth Public Clients (RFC 7636)

ICAP 內容驗證整合

本文件說明 MaiAgent 平台如何整合 ICAP (Internet Content Adaptation Protocol) 協定,實現對使用者上傳檔案和對話內容的安全掃描與驗證,防範惡意軟體和不當內容。

1. 什麼是 ICAP？

ICAP (Internet Content Adaptation Protocol) 是一個輕量級的協定,用於將內容轉發給外部服務進行處理,如病毒掃描、內容過濾、資料外洩防護等。

1.1 ICAP 的核心價值

考量面向	無內容驗證方案	ICAP 內容驗證方案
安全性	使用者可能上傳惡意軟體或不當內容	自動掃描並阻擋惡意檔案和內容
合規性	難以符合企業資安政策要求	整合企業級防毒和 DLP 解決方案
靈活性	需自行開發內容檢測邏輯	使用標準協定整合第三方專業服務
維護性	需持續更新病毒碼和檢測規則	由專業廠商負責維護檢測引擎

1.2 常見應用場景

檔案上傳安全: 使用者上傳到知識庫的文件需先經過病毒掃描

對話內容過濾: 檢測對話中是否包含不當言論、敏感資訊

資料外洩防護 (DLP): 防止使用者上傳包含機密資訊的檔案

合規性檢查: 確保內容符合 GDPR、HIPAA 等法規要求

2. MaiAgent ICAP 整合架構



Mermaid 圖表

2.1 核心元件說明

ICAP 客戶端: MaiAgent 內建的 ICAP 協定實作,負責與 ICAP 伺服器通訊

ICAP 伺服器: 第三方內容掃描服務,如 Symantec、McAfee、Trend Micro 的防毒閘道器

處理佇列: 非同步處理檔案上傳和掃描任務,避免阻塞使用者操作

掃描記錄: 儲存所有掃描結果,包括檔案雜湊、掃描時間、結果等資訊

2.2 ICAP 請求流程



Mermaid 圖表

3. ICAP 協定實作細節

3.1 ICAP 請求格式

MaiAgent 使用 REQMOD (Request Modification) 模式發送檔案給 ICAP 伺服器:

```
REQMOD icap://icap-server.example.com/scan ICAP/1.0



POST /upload HTTP/1.1



400
[Binary file content in chunked encoding]
0
```

關鍵欄位說明:

REQMOD: 請求修改模式,用於在內容儲存前進行檢查
Encapsulated: 描述 HTTP 請求的封裝格式
Allow: 204: 告知 ICAP 伺服器如果內容無需修改可直接返回 204
Chunked Encoding: 使用分塊傳輸編碼發送檔案內容

3.2 分塊傳輸編碼 (Chunked Transfer Encoding)

MaiAgent 正確實作了 HTTP 分塊傳輸編碼:

```
分塊傳輸格式:
<資料大小（十六進位）>\r\n
<資料內容>\r\n
...
0\r\n\r\n （結束標記）
```

為何需要正確的分塊編碼？

ICAP 協定要求使用 HTTP 分塊傳輸編碼來傳送請求和回應本體
錯誤的編碼格式會導致 ICAP 伺服器無法正確解析檔案內容
MaiAgent 修正了分塊大小的十六進位格式表示,確保與 ICAP 標準相容

3.3 ICAP 回應處理

ICAP 伺服器可能返回以下回應:

狀態碼	含義	MaiAgent 處理方式
200 OK	內容已檢查或修改	接受修改後的內容(如有),儲存檔案
204 No Content	內容無需修改	直接儲存原始檔案
403 Forbidden	內容被禁止(如包含病毒)	拒絕儲存,向客戶端發送 ICAP_BLOCKED 事件
500 Internal Error	ICAP 伺服器錯誤	記錄錯誤,向客戶端發送 ICAP_ERROR 事件

4. WebChat 整合

當 ICAP 掃描完成後，系統會即時通知使用者掃描結果：

檔案被阻擋: 顯示友善的錯誤訊息，說明檔案被拒絕的原因（如偵測到病毒）
系統錯誤: 提示使用者稍後再試，並記錄錯誤日誌

使用者體驗優化

即時回饋: 上傳檔案後立即顯示「掃描中」狀態
進度指示: 大型檔案上傳時顯示進度條
友善錯誤訊息: 使用易懂的語言說明檔案被拒絕的原因
重試機制: 如果是暫時性錯誤,提供重新上傳選項

5. 掃描記錄管理

5.1 記錄保存

MaiAgent 會儲存所有 ICAP 掃描的詳細記錄:

每筆掃描記錄包含以下資訊:

欄位	說明
檔案識別碼	被掃描檔案的唯一識別碼
檔案名稱	原始檔案名稱
檔案雜湊值	SHA-256 雜湊，用於確認檔案完整性
掃描結果	PASS（通過）、BLOCKED（阻擋）、ERROR（錯誤）
掃描原因	當檔案被阻擋時，記錄具體原因

欄位	說明
ICAP 伺服器	執行掃描的 ICAP 伺服器位址
掃描時間	掃描執行的時間戳記
使用者 組織	發起掃描的使用者與所屬組織

5.2 過期記錄清理

MaiAgent 實作了自動清理機制,定期刪除過期的掃描記錄:

系統會透過背景排程任務，自動清理超過保留期限的掃描記錄。

清理策略:

保留期限: 依組織 ICAP 設定的「記錄保留天數」，預設 30 天；設為 0 表示永久保留

適用範圍: 保留天數以組織為單位，同一組織的所有掃描記錄套用同一個期限

6. 配置與部署

6.1 ICAP 伺服器配置

ICAP 由 MaiAgent 在系統管理端依組織設定，管理後台沒有設定畫面；需要啟用或調整時，請洽 MaiAgent 窗口並提供 ICAP 伺服器資訊。每個組織可設定的項目如下：

設定項目	說明
ICAP 伺服器網址	ICAP 服務的連線 URL，設定時可先測試連線
是否啟用	開啟後才會檢查；另外，Agent 使用的模型也要由 MaiAgent 開啟 ICAP 檢查，兩者都開才會生效
阻擋訊息	內容被攔截時顯示給使用者的訊息
記錄保留天數	掃描記錄的保留期限，預設 30 天，0 表示永久保留

6.2 支援的 ICAP 服務

MaiAgent 相容以下主流 ICAP 服務:

Symantec Protection Engine: 企業級防毒和內容過濾

McAfee Web Gateway: 閘道層級的內容掃描

Trend Micro IWSVA: 整合式 Web 安全與內容掃描

Kaspersky Scan Engine: Kaspersky 的 ICAP 掃描引擎
ClamAV (透過 c-icap): 開源防毒解決方案

6.3 效能考量

為確保系統效能,建議:

非同步掃描: 大型檔案使用背景任務掃描,不阻塞上傳回應
快取機制: 對相同檔案(相同雜湊值)的掃描結果進行快取
ICAP 連線池: 維護與 ICAP 伺服器的持久連線,減少連線開銷
負載均衡: 使用多個 ICAP 伺服器實例分散掃描負載

7. MaiAgent ICAP 整合的技術優勢

7.1 安全性優勢

多層防護: 在檔案儲存前就進行掃描,防範惡意內容
即時阻擋: 發現威脅立即阻擋,不會儲存到系統中
專業引擎: 整合業界領先的防毒和 DLP 引擎
完整記錄: 所有掃描活動都有詳細記錄,便於安全稽核

7.2 合規性優勢

標準協定: 使用業界標準 ICAP 協定,易於整合現有企業安全設施
稽核軌跡: 完整的掃描記錄符合合規要求
靈活配置: 可根據不同產業的合規需求調整掃描策略
資料保護: 支援 DLP 引擎,防止敏感資料外洩

7.3 運維優勢

解耦架構: ICAP 掃描邏輯與應用程式解耦,易於維護
水平擴展: 可輕鬆增加 ICAP 伺服器處理更多掃描請求
統一管理: 企業可使用現有的 ICAP 基礎設施,統一管理所有應用的內容掃描
詳細日誌: 完整的掃描日誌便於問題排查和效能優化

8. 相關文檔

AI 安全防護機制 - 了解 MaiAgent 的多層次安全防護策略

檔案上傳與知識庫管理

參考連結

ICAP RFC 3507

Chunked Transfer Encoding (RFC 7230)

Open ICAP Forum

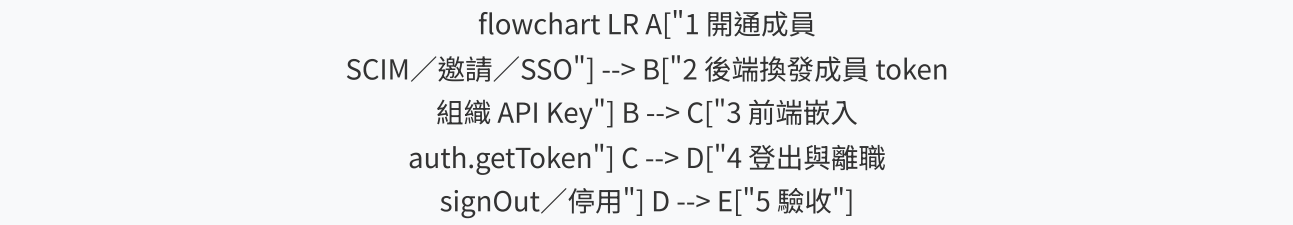
MaiAgent Work｜員工生產力整合

把 MaiGPT 加進您的系統

這一頁給「我們的系統要加上 MaiGPT」的工程師從頭讀到尾。不管您是企業自己的入口網站、內部工作台，還是替客戶代管的 SaaS，只要使用者是 **MaiAgent 組織的成員**（員工、承辦人），流程都一樣。每一步把要做的事直接寫在這裡，想深入才點連結。

> 如果您的使用者是您的客戶、會員或匿名訪客、不是組織成員，MaiGPT 產品本身不能給他們用，請改走自建 AI Agent＋MaiGPT 視窗模式，聯絡人身分對接見軟體廠商整合指南。

全貌



Mermaid 圖表

做完之後：員工在您的系統登入，頁面裡直接出現 MaiGPT，能生圖、Deep Research、用連接器，權限、額度、對話歷史都跟著他在 MaiAgent 的成員身分；他不需要另外登入 MaiAgent，也不需要知道 MaiAgent 的存在。

開始前準備

項目	到哪拿
一個 MaiAgent 組織，方案含 MaiGPT 與身分同步	不確定請洽業務
組織 API Key，持有者是組織擁有者或具「組織總覽」管理權限	取得方式見快速上手
MaiGPT 的 Web Chat ID	`GET /api/v1/web-chats/` 回傳中名稱為 `MaiGPT`、`accessType` 為 `web` 的那一筆
角色先建好、權限先設好	後台「組織設定 → 組織總覽 → 編輯角色權限」建例如「員工」角色，勾選 MaiGPT 使用權（設定方式）。**成員的能力全部來自角色**，這步沒做，後面全部白做

項目	到哪拿
決定成員從哪來	下一節三選一

第一步：開通成員

您的使用者要先是 MaiAgent 組織的成員。三種做法，選最省事的那個：

> 組織裡已經有人在用 MaiAgent？接 SCIM 不會讓他們原本的帳號失效，也不會建立重複帳號；被接管的既有成員之後會跟著您的系統同步。細節見既有帳號與成員會怎樣。

您的系統有使用者管理（推薦）

在您系統新增、修改、停用使用者的事件裡，順手呼叫 MaiAgent 的 SCIM 端點，成員即時一致。

後台「組織設定 → 身份同步」選 SCIM，「連線資訊」分頁複製 Endpoint URL 與 Token（Token 只顯示一次，放伺服器環境變數）
把即時同步頁的 `MaiAgentMembers` client 放進您的專案，在事件裡呼叫：

```
members = MaiAgentMembers()   # 讀環境變數 MAIAGENT_SCIM_URL  MAIAGENT_SCIM_TOKEN

# 新增使用者
user.maiagent_member_id = members.on_user_created(
    external_id=user.id, email=user.email, name=user.display_name, groups=["員工"],
)
# 改名、換部門
members.on_user_updated(user.maiagent_member_id, user.id, user.email,
user.display_name, ["員工"])
# 停權、離職
members.on_user_deactivated(user.maiagent_member_id)
```

第一次上線先把既有使用者跑一輪 `on_user_created`（冪等，批次補建），之後靠事件維持。

只有名單（HR 匯出、Excel、資料庫）

用名單同步工具：準備一份四欄的 CSV 或 JSON（人員編號、Email、姓名、角色），設好 SCIM 連線環境變數，先 `--dry-run` 看計畫再真跑，之後排程每天跑一次。名單上有的建立或更新、名單上消失的停用。

有 Okta／Entra ID／JumpCloud

把 SCIM 的 Endpoint URL 與 Token 填進 IdP 的 provisioning 設定，IdP 會自動推送使用者與群組。細節見從 IdP 推送。

三件事要記得，都是實際整合時最常卡的：

角色名稱對上後台建好的角色：SCIM 建成員時 `groups` 填的名稱，就是後台那個已設好權限的角色；名稱對不上會自動建一個**沒有權限**的新角色。

Email 是帳號的鍵：`userName` 就是之後換發 token 用的 Email；建立後不能經 SCIM 改 Email。

SCIM 建的帳號沒有密碼：這條路的員工不會登入 MaiAgent，這正是我們要的；若同時要開放 MaiGPT 網頁版或 App，另設 SSO。

第二步：後端換發成員 token

在您的後端開一支只給**已登入使用者**呼叫的端點（例如 `GET /maigpt-token`），內部以組織 API Key 向 MaiAgent 換發該成員的短效 token：

```
curl -X POST https://api.maiagent.ai/api/v1/members/embed-tokens/
  -H "Authorization: Api-Key <您的組織 API Key>"
  -H "Content-Type: application/json"
  -d '{"email": "alice@acme.com"}'
```

成功回 `201`：

```
{
  "accessToken": "<成員短效 JWT>",
  "organizationId": "8d2f … - … ",
  "expiresAt": 1789906401000,
  "member": { "id": "c1a4 … ", "userId": "5e77 … ", "email": "alice@acme.com",
"name": "Alice" }
}
```

把整份回應原樣回給前端就好。規則與狀態碼：

事實	說明
效期 15 分鐘、不發 refresh token	續期由前端 SDK 自動處理（第三步），您的後端只要「被叫到就換一顆」——**每次都換新的，不要快取**
token 只能用在 MaiGPT 對話端點	拿去打 `/api/v1/` 任何 API 一律 `401`，外洩也不能碰組織資料
`401` ／ `403`	沒用 Api-Key／Key 的持有者不是擁有者或組織管理員
`404`	這個 Email 不是本組織的啟用成員——第一步沒做、或已被停用

事實	說明
每次換發都留稽核紀錄	組織的 API 請求紀錄可查，回應中的 token 不會被記錄

參數與更多細節見成員 token 換發。

第三步：前端嵌入

在要放 MaiGPT 的頁面加入嵌入碼，`auth.getToken` 指向第二步的端點；其餘都交給 SDK（初始化時取一次 token、到期前自動再取、同一則對話續用）：

```
<div id="maigpt-container" style="height: 720px;"></div>
<script>
  window.maiagentChatbotConfig = {
    webChatId: '<MaiGPT 的 Web Chat ID>',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    enabledWindowModes: ['maigpt'],
    targetElement: '#maigpt-container',
    maigptTitle: 'Acme GPT',            // 選填：左上角品牌名
    locale: 'zh-TW',                    // 選填：介面語系
    auth: {
      getToken: async () => {
        const res = await fetch('/maigpt-token', { credentials: 'include' })
        const { accessToken, organizationId, member } = await res.json()
        return { accessToken, organizationId, sourceId: member.userId }
      },
    },
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

> sourceId 一定要填回應裡的 member.userId，不是 member.id、也不是您系統的使用者編號。MaiGPT 靠它把對話對應到成員，填錯會變成匿名對話、看不到該成員的歷史。

容器要自己給高度；SPA 要等容器 mount 後再載入 SDK；`baseUrl` 與 SDK 網址是 SaaS 位址，私有部署請換成您的網域。版面與其他參數見 MaiGPT 模式嵌入。

第四步：登出與離職

事件	您要做的事	結果
使用者登出您的系統	前端呼叫 `MaiAgent.auth.signOut()`	iframe 清掉身分並停止續期；已發出的 token 最多再存活 15 分鐘
使用者離職／停權	第一步的停用（`on_user_deactivated`、名單移除、IdP 停用）	之後換發回 `404`，嵌入頁自然失效；對話與額度紀錄保留
復職	第一步的重新啟用	角色與歷史恢復

第五步：驗收

依序確認，每一項都有明確的判斷點：

成員存在且有角色：後台「成員管理」看得到這個人，角色是您設好權限的那個。

換發成功：第二步 curl 回 `201`；換一個不存在的 Email 回 `404`。

能對話：嵌入頁開啟 MaiGPT、送一句話得到回覆；用該成員登入 MaiGPT 網頁版（若有 SSO）或後台對話紀錄能看到這則對話。

能力對：請它生一張圖；若角色有開連接器與 Deep Research，入口會出現。

續期無感：頁面停留超過 15 分鐘再送訊息，不用重新整理、同一則對話繼續。

登出生效：`signOut()` 後再送訊息被拒；停用該成員後第二步回 `404`。

上線後

SCIM Token 與 API Key 輪換：SCIM Token 建立超過 180 天後台會提醒；重新產生後舊的立即失效，記得同步更新伺服器環境變數。

方案容量：成員數影響內建 Agent 席次與每成員 Credits 額度，見成員 Credits 額度管理。

稽核：每次換發 token 都在組織的 API 請求紀錄；SCIM 的每次同步在後台「身份同步 → 同步紀錄」。

出錯時

現象	原因	處理
換發回 `404`	Email 不是本組織啟用成員	回第一步確認成員存在、未停用、Email 相同（大小寫不分）
換發回 `403`	API Key 持有者不是擁有者也沒有組織管理權限	換用管理員發的 Key
聊天視窗「建立對話失敗（400）您沒有權限使用	成員的角色沒有 MaiGPT 使用權，或 `auth` 沒帶到	後台角色權限開 MaiGPT；確認 `getToken` 有回 `accessToken` 與 `organizationId`

現象	原因	處理
MaiGPT」		
對話是匿名的、看不到歷史	`sourceId` 填錯	用 `member.userId`
15 分鐘後送訊息失敗	`getToken` 沒被呼叫或失敗（看瀏覽器 console 的 `[MaiAgent]` 錯誤）	確認端點對已登入使用者可用、回應格式正確
成員建好但什麼都不能用	沒進任何角色，或角色權限是空的	後台角色權限設定；SCIM 用相同名稱指派
改了 Email 沒生效	SCIM 不允許變更 Email	後台修改，或停用舊帳號以新 Email 建立

更多 SCIM 錯誤碼見名單同步工具的錯誤對照。

深入閱讀

成員開通：SCIM／邀請／SSO — SCIM API 完整參考、三條路對照
即時同步：在使用者事件中呼叫／名單同步工具
串接 MaiGPT：選對 Agent、補齊能力 — MaiGPT 產品 vs 自建 Agent、成員身分嵌入的完整參數
Web Chat 嵌入與 SDK／MaiGPT 模式嵌入

串接 MaiGPT：選對 Agent、補齊能力

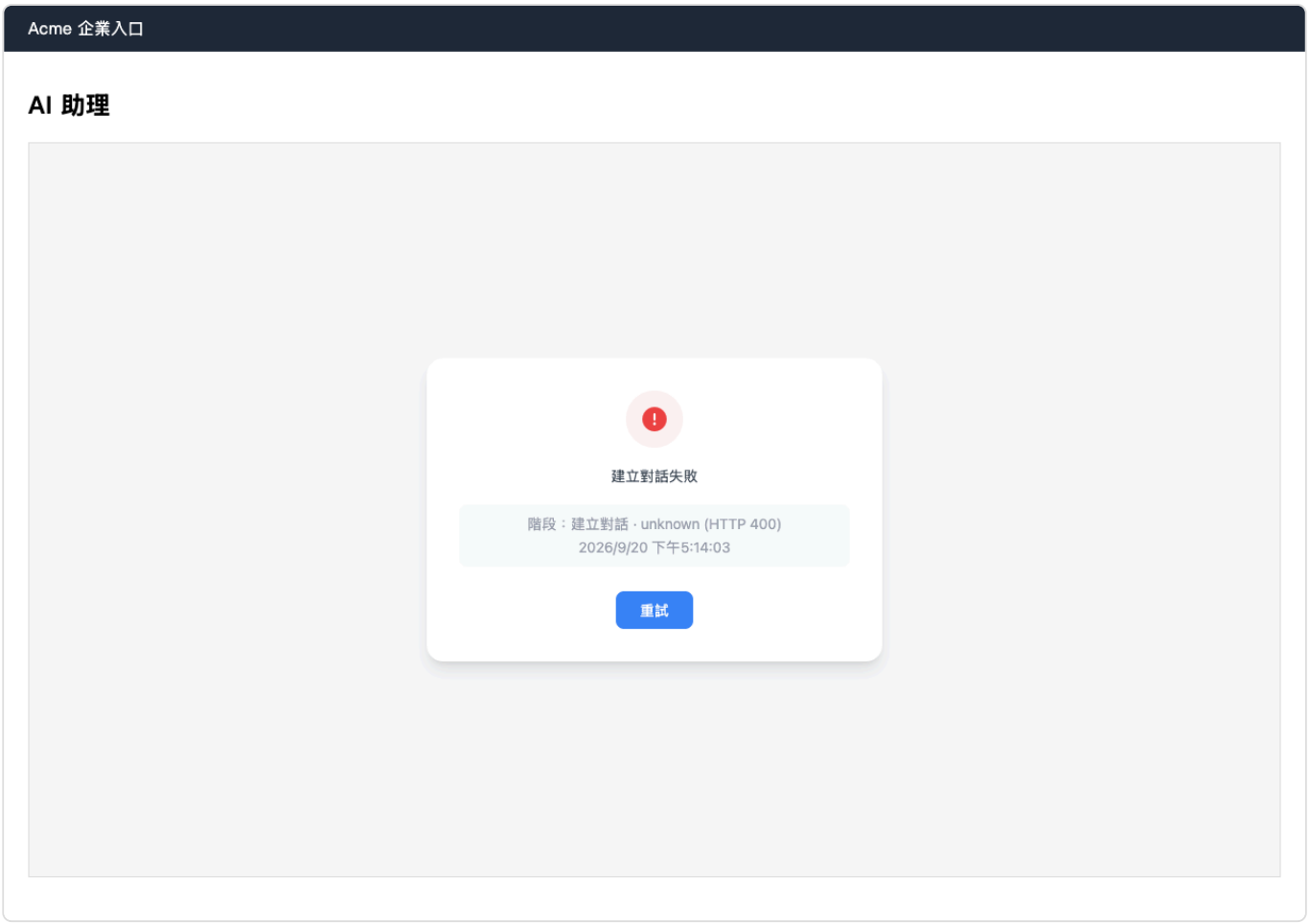
「MaiGPT」在 MaiAgent 平台上指兩件不同的東西，串接前先分清楚，可以省掉大部分的來回：

	MaiGPT 產品（組織內建 Agent）	**MaiGPT 視窗模式（Web Chat SDK 的一種佈局）**
是什麼	每個組織自動配備一個、由 MaiAgent 統一維護的企業 AI 工作台，內建 AI 生圖、Deep Research、連接器等能力，見什麼是 MaiGPT？	`enabledWindowModes: ['maigpt']` 開啟的**類 ChatGPT 完整介面**（側欄對話歷史、搜尋、設定），純前端呈現
誰能用	組織**成員**登入後使用（網頁、iOS App、LINE、Microsoft Teams），權限由角色使用權限控制	任何 Web Chat 都能套用，含匿名訪客與聯絡人身分
能不能嵌進您的系統	**只能給組織成員用**：建立對話時要求成員身分且具備 `maigpt_access` 權限，直接用其 Web Chat ID 嵌入會得到 `400 您沒有權限使用 MaiGPT。`；要嵌給成員用，走成員身分嵌入由您的後端換發成員 token	**能**：本頁第二、三節講的就是這件事
能不能客製化	**不能**：指令、模型、工具、技能皆由平台統一設定，組織端只能透過角色權限**收窄**	**能**：介面下面掛的是您自己建立的 AI Agent，指令、模型、工具、知識庫全由您決定

結論：分兩種需求。給**自家客戶、非組織成員**用，做法是**自己建立一個 AI Agent，補上需要的工具，再用 MaiGPT 視窗模式嵌入**（第二、三節）。給**組織成員**用、而且就是要 MaiGPT 產品本身的能力（生圖、Deep Research、連接器、角色權限），走成員身分嵌入（第五節），由您的後端替成員換發短效 token。

一、常見症狀對照

您看到的現象	**原因**	**解法**
介面長得像 MaiGPT，但問答像一般 Chat，要求生圖時 AI 回「我無法生成圖片」	嵌入的是一般 AI Agent，且（1）回答模式不是 **Agent 模式**，或（2）沒有掛上圖片生成等工具	依第二節切換 Agent 模式並加上工具
聊天視窗打不開，顯示「建立對話失敗（HTTP 400）」，API 回 `您沒有權限使用 MaiGPT。`	嵌入的 Web Chat ID 屬於組織內建的 MaiGPT 產品，而訪客沒有成員身分	給客戶用 → 改用自建 Agent 的 Web Chat ID（第三節）；給成員用 → 依成員身分嵌入帶入成員 token
後台的 AI Agent 列表 API 找不到 MaiGPT	`GET /api/v1/chatbots/` 只列出組織自建的 Agent，MaiGPT 產品不在其中	這是預期行為；串接一律以自建 Agent 為對象



嵌入組織內建 MaiGPT 的 Web Chat：訪客不是登入成員，建立對話被拒（HTTP 400）

二、建立能生圖、能用工具的 AI Agent

以下用「AI 生圖」為例，其他能力（網頁搜尋、網頁讀取、MCP 工具、您自家的 API 工具）加法相同。

建立 AI Agent

後台「AI 功能 → AI 助理」建立一個新的 AI Agent，填好角色指令與模型。操作見使用者手冊建立 Agent。

回答模式切換為「Agent 模式」

進入該 Agent 的「回答模式設定」，將回答模式改為 **Agent 模式**。

> 只有 Agent 模式會調用工具。預設的「RAG 問答」模式反應快，但不會使用任何工具——這是「明明加了生圖工具卻還是不生圖」最常見的原因。

加上工具

進入該 Agent 的「工具」頁，點「選擇工具」，勾選需要的內建工具，例如：

圖片生成：`Global_Image_Tool_OpenAI_GPT_Image_2`、`Global_Image_Tool_Gemini_3` 等（勾一個即可；模型差異見內建 AI 圖像生成工具）

網頁搜尋：`Global_Tavily_Search_Tool`

網頁讀取：`Global_Web_Fetch_Tool`

您自家系統的 API 工具或 MCP 工具

完整操作見使用者手冊為 Agent 配置工具。

建立 Web Chat 對話平台

「客服對話 → 對話平台」為這個 Agent 新增一個 **Web Chat**，取得 `webChatId`。操作見串接對話平台：網站。

用 API 自動化（ISV 代客建置時）

以上四步都有對應 API，適合軟體廠商在為每個客戶開通時以程式完成。認證方式與 API Key 取得見快速上手。

```
# 1. 列出內建工具，取得圖片生成工具的 id
GET /api/v1/tools/

# 2. 建立 AI Agent：回答模式 agent ＋ 掛上工具
POST /api/v1/chatbots/
{
  "name": "Acme GPT",
  "instructions": "你是 Acme 公司的 AI 助理，請用繁體中文回答。",
  "largeLanguageModel": "<LLM id>",
  "embeddingModel": "<embedding model id>",
  "rerankerModel": "<reranker model id>",
  "replyMode": "agent",
  "tools": ["<Global_Image_Tool_OpenAI_GPT_Image_2 的 id>"]
}

# 3. 為 Agent 建立 Web Chat 對話平台（回應的 channel.id 就是 webChatId）
POST /api/v1/inboxes/
{
  "name": "Acme GPT",
  "chatbot": "<chatbot id>",
  "channelType": "web",
  "channel": {
    "name": "Acme GPT",
    "theme": { "primaryColor": "#1890ff", "...": "..." },
    "darkTheme": { "primaryColor": "#404040", "...": "..." }
  }
}
```

既有 Agent 改模式或補工具用 `PATCH /api/v1/chatbots/{id}`，`tools` 傳完整的工具 id 陣列（整份取代）

`theme` 與 `darkTheme` 為必填，欄位清單可先 `GET /api/v1/web-chats/{既有 webChatId}` 取一份現成的複製

Web Chat 一定要透過 `POST /api/v1/inboxes/` 連同收件匣一起建立；直接 `POST /api/v1/web-chats/` 會建出沒有收件匣的孤兒 Web Chat，嵌入時回 404

三、用 MaiGPT 視窗模式嵌入

拿第二節的 `webChatId`，照 MaiGPT 模式嵌入的嵌入碼放進您的頁面：

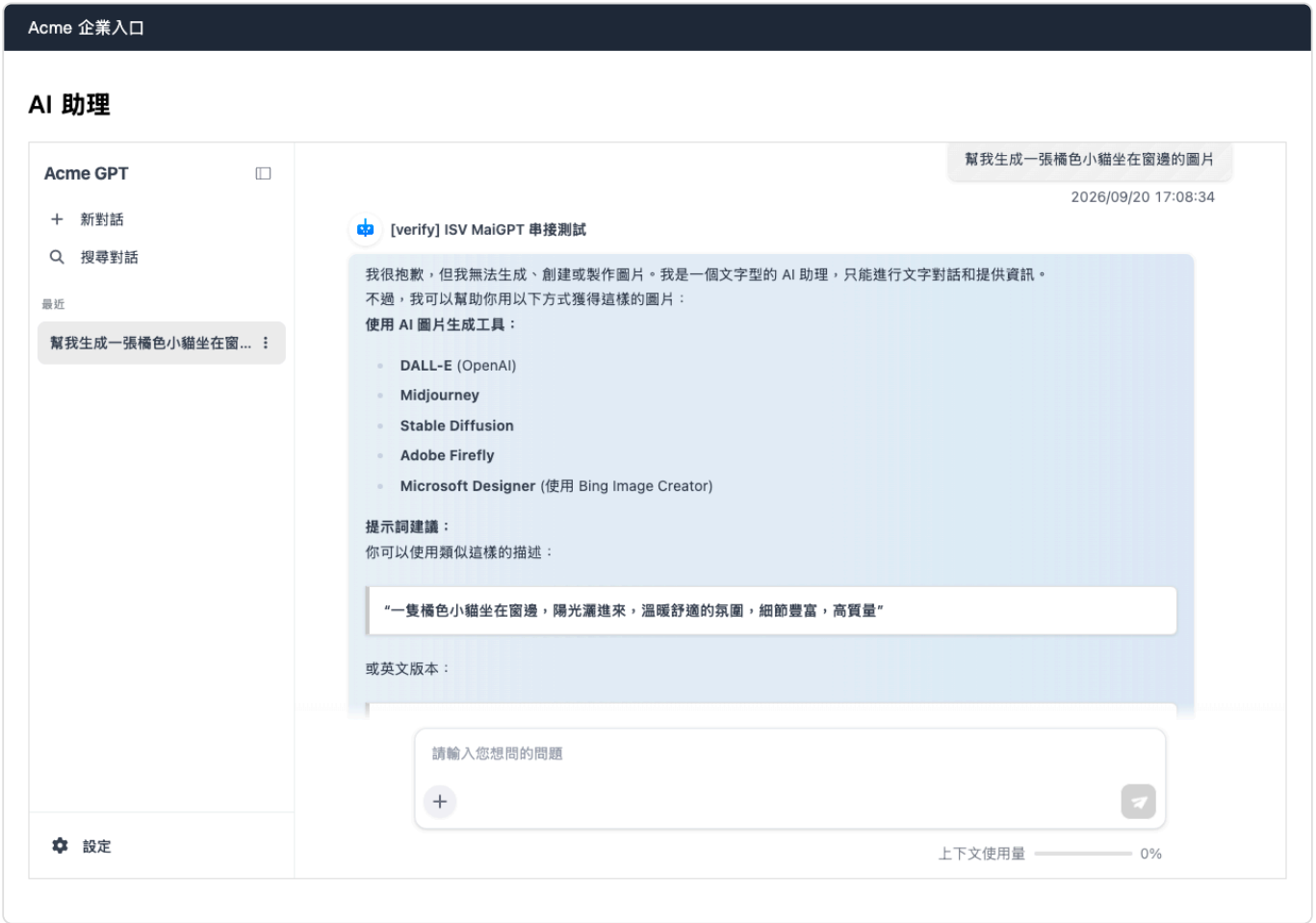
```
<div id="maigpt-container" style="height: 720px;"></div>
<script>
  window.maiagentChatbotConfig = {
    webChatId: '<第二節建立的 webChatId>',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    enabledWindowModes: ['maigpt'],
    targetElement: '#maigpt-container',
    maigptTitle: 'Acme GPT',
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

要讓 Agent 認得登入使用者、以其權限呼叫您的 API，再依軟體廠商整合指南加上 `contactId` 或 `auth`。

驗證：同一段嵌入碼，補齊能力前後

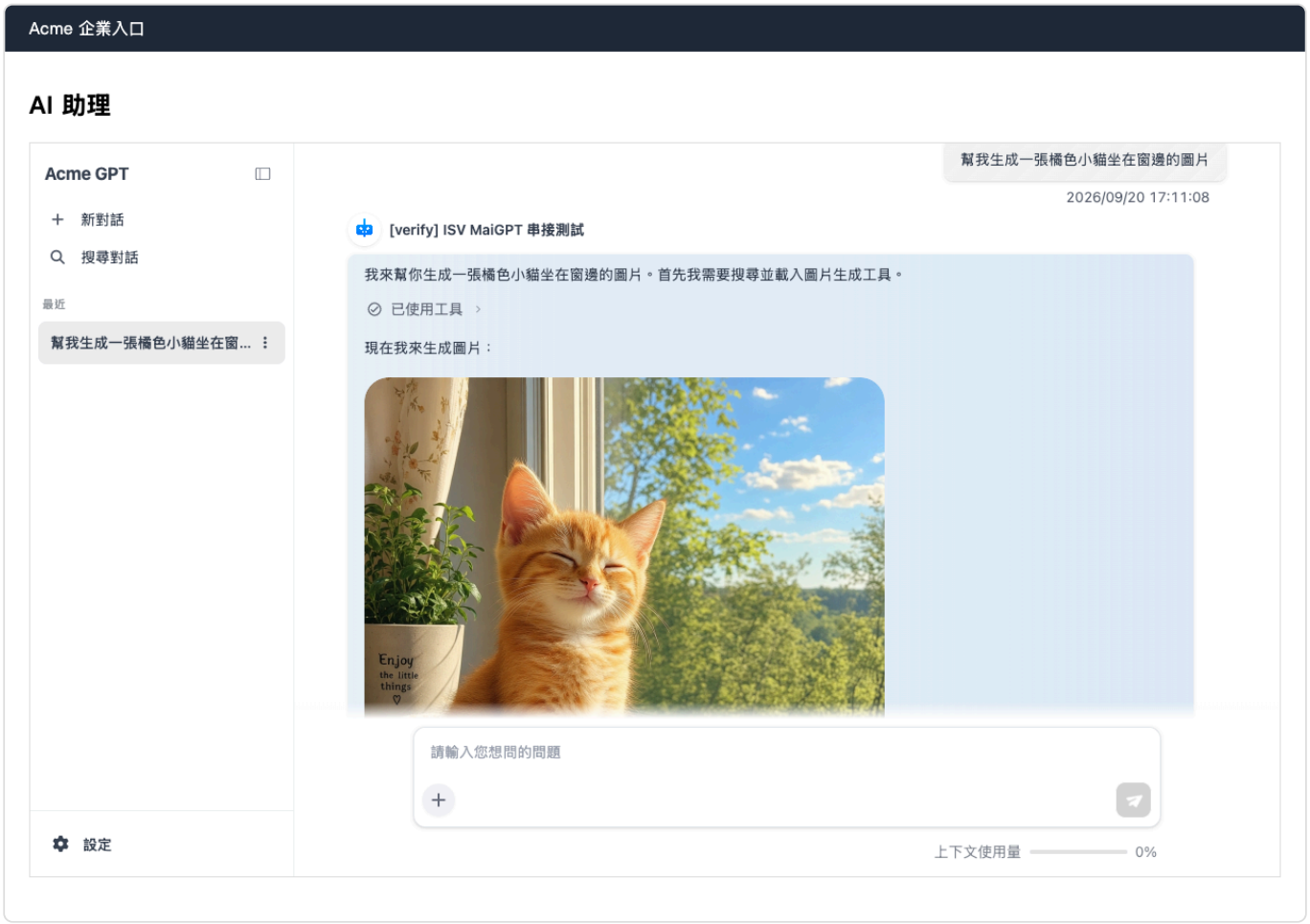
下面兩張圖是同一個 AI Agent、同一段嵌入碼，只差「回答模式＋工具」的設定：

補齊前：RAG 問答模式、沒有工具



介面是 MaiGPT 視窗模式，但 Agent 回「我無法生成、創建或製作圖片」

補齊後：Agent 模式＋圖片生成工具



同一段嵌入碼，Agent 呼叫圖片生成工具並在對話中直接出圖

四、什麼時候該用 MaiGPT 產品本身

如果需求是「讓我們公司的員工有一個 ChatGPT 等級的內部 Agent」，而不是「嵌進自家產品給自家客戶用」，直接用 MaiGPT 產品最省事：

員工以組織成員身分登入平台即可使用，入口有網頁版、iOS App、LINE、Microsoft Teams
生圖、Deep Research、連接器、技能都已內建，管理員只需用角色使用權限決定誰能用什麼
想加公司專屬知識：把知識庫掛給成員可見的範圍即可，不需要另建 Agent

「嵌到自家系統但使用者是組織成員」→ 走第五節的成員身分嵌入。只要出現「給非成員（客戶、會員）用」「要加自家 API 工具」「要改指令或模型」其中任何一項，就走第二、三節的自建 Agent 路線。

五、成員身分嵌入：把 MaiGPT 產品嵌進自家系統給組織成員

> 想從頭到尾照著做，請看把 MaiGPT 加進您的系統；本節是各參數的完整參考。

適用情境：您的系統使用者**同時是 MaiAgent 組織的成員**（例如企業入口網站、員工工作台、ISV 代管的企業內部系統），而且要的就是 MaiGPT 產品本身，不想另建 Agent。做法是由**您的後端**以組織 API Key 替登入中的成員換一顆短效 token，前端用 SDK 既有的成員注入設定嵌入 MaiGPT 的 Web Chat。



Mermaid 圖表

前置條件

一把**組織 API Key**，且持有者是組織擁有者或角色具「組織總覽」管理權限——只有能管理成員的人發的 Key 才能代表成員換發。API Key 取得見快速上手

要嵌入的使用者已是該組織成員，且角色具 MaiGPT 使用權限（MaiGPT 角色使用權限）；成員不存在時換發會回 404，不會自動開通。人員怎麼變成成員（SCIM 開通 API、後台邀請、SSO）見成員開通，ISV 代管的標準接法是 SCIM 建成員＋本節換發 token

MaiGPT 的 Web Chat ID：`GET /api/v1/web-chats/` 回傳中名稱為 `MaiGPT`、`accessType` 為 `web` 的那一筆

步驟一：後端換發成員 token

在您的後端開一支只給登入使用者呼叫的端點（例如 `GET /maigpt-token`），內部轉呼叫 MaiAgent；前端的 SDK 會在需要時打它：

```
curl -X POST https://api.maiagent.ai/api/v1/members/embed-tokens/
  -H "Authorization: Api-Key <您的組織 API Key>"
  -H "Content-Type: application/json"
  -d '{"email": "alice@acme.com"}'
```

`email` 與 `memberId` 二擇一（`memberId` 為 MaiAgent 成員 ID）。成功回 `201`（`expiresAt` 為 epoch 毫秒）：

```
{
  "accessToken": "<成員短效 JWT>",
  "organizationId": "8d2f ... - ... ",
  "expiresAt": 1789906401000,
  "member": {
    "id": "c1a4 ... - ... ",
    "userId": "5e77 ... - ... ",
    "email": "alice@acme.com",
    "name": "Alice"
  }
}
```

狀態碼	意思
`400`	`email` 與 `memberId` 兩個都給或都沒給
`401`	沒有以 Api-Key 認證（成員登入 token、匿名都會被拒）
`403`	API Key 持有者不是組織擁有者、也沒有組織管理權限
`404`	指定的成員不存在、已停用、或不屬於這把 Key 的組織

> token 規則：效期固定 15 分鐘、不發 refresh token、不可延長；同一成員可重複換發，舊 token 到期前仍有效。這顆 token 只能用於 Web Chat 對話（建立對話、送訊息、交接續談三個呼叫），拿去呼叫 /api/v1/ 任何端點、其他對話管理動作或 WebSocket 一律 401。每次換發都會留在組織的 API 請求紀錄中供稽核（回應中的 token 不會被記錄）。

步驟二：前端給 SDK 一個取 token 的函式

嵌入設定與第三節相同，多一個 `auth.getToken`；`webChatId` 換成 MaiGPT 的 Web Chat ID：

```
<div id="maigpt-container" style="height: 720px;"></div>
<script>
  window.maiagentChatbotConfig = {
    webChatId: '<MaiGPT 的 Web Chat ID>',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    enabledWindowModes: ['maigpt'],
    targetElement: '#maigpt-container',
    auth: {
      // SDK 初始化時呼叫一次；之後每當 token 快到期會再呼叫一次並自動續期，您不需要計時
      getToken: async () ⇒ {
        const res = await fetch('/maigpt-token', { credentials: 'include' })  // 您的後端，見步驟一
        const { accessToken, organizationId, member } = await res.json()
        return { accessToken, organizationId, sourceId: member.userId }
      },
    },
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

`getToken` 回傳的物件欄位：

欄位	必填	說明
`accessToken`	✅	步驟一回應的 `accessToken`
`organizationId`	✅	步驟一回應的 `organizationId`
`sourceId`	建議	步驟一回應的 `member.userId`；也可以改寫在 `auth.sourceId`，兩處都給時以 `auth.sourceId` 為準

`auth` 必須寫在 `maiagentChatbotConfig` 裡，SDK 建立聊天視窗時才知道要先等成員身分注入完成再建立第一則對話；改在 SDK 載入後才呼叫 `MaiAgent.auth.setup()` 的話，第一則對話可能在身分注入前就以匿名建立而被拒。

> sourceId 必須是回應中的 member.userId，不是 member.id、也不是您系統的使用者編號。MaiGPT 以它把對話對應到成員；填錯會變成匿名對話、看不到該成員的 MaiGPT 歷史。

到期與續期怎麼運作

token 效期 15 分鐘。iframe 會在到期前 60 秒通知 SDK，SDK 再呼叫一次 `getToken` 取新 token 並重新注入，**同一則對話直接續用，不重載頁面或 iframe**，使用者無感。

`getToken` **每次都必須向 MaiAgent 換發新 token，不可回傳快取值**：SDK 依 token 自身的存活時間排程下一次續期；回傳與上一顆相同的 token 會被視為失敗，走下一條的重試流程。

`getToken` 失敗（例如您的使用者已登出、後端回 401）時 SDK 會在 30 秒、60 秒、120 秒後各重試一次，仍失敗就停止並在 console 記錄錯誤；token 到期後下一則訊息會送出失敗並顯示既有的錯誤提示，對話與已送出的訊息不會消失。

使用者登出您的系統時呼叫 `MaiAgent.auth.signOut()`，SDK 會清掉身分並停止續期；已發出的 token 最多再存活 15 分鐘。換另一位使用者登入時直接再呼叫一次 `MaiAgent.auth.setup()` 即可，前一位使用者進行中的續期會被作廢。

該 Web Chat 若開啟了 Source ID 簽章驗證：簽章是一次性的，SDK 續期時不會重用第一次的 `ts` ／ `sig`，請讓 `getToken` 的回傳值連新簽章一起帶回（`{ accessToken, ts, sig }`）。

若您不用 `getToken`、改直接給靜態的 `auth.accessToken`，SDK 不會續期，15 分鐘後要自己重新呼叫 `MaiAgent.auth.setup()`。

這條路與自建 Agent 怎麼選

	成員身分嵌入 MaiGPT（本節）	**自建 Agent＋MaiGPT 視窗模式（第二、三節）**
使用者	必須是組織成員	任何人（客戶、會員、匿名）
能力	MaiGPT 全套：生圖、Deep Research、連接器、技能，依角色權限	您勾選的工具與知識庫
客製指令／模型	不能（平台統一）	能
後端工作	一支轉呼叫換發 API 的端點	零（可選加聯絡人身分同步）
計費歸屬	該成員的額度	該 Agent／Web Chat

六、常見問題

- ▶ 我在後台「對話平台」看到一個叫 MaiGPT 的 Web Chat，可以拿它的嵌入碼嗎？
- ▶ MaiGPT 產品裡的生圖、Deep Research、連接器，自建 Agent 都能有嗎？
- ▶ 回覆模式已經是 Agent、也掛了生圖工具，還是不生圖？
- ▶ MaiGPT 視窗模式跟 floating／sidebar 差在哪？

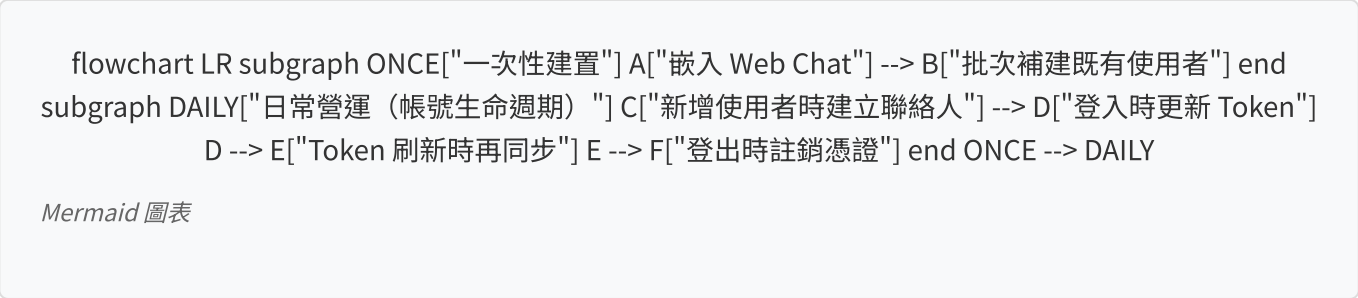
MaiAgent Serve｜客戶服務整合

軟體廠商整合指南：把 AI Agent 嵌給您的客戶

本頁適合「把 MaiAgent Web Chat 嵌入自家產品、並與自家帳號系統做權限對接」的軟體廠商：使用者在您的系統登入後，AI Agent 認得他是誰、能回溯他的對話歷史，並以**他的權限**呼叫您系統的 API。

整合旅程共五個階段，本頁說明每個階段做什麼、按什麼順序做，細節分別連結到對應的技術文件。

整合全景



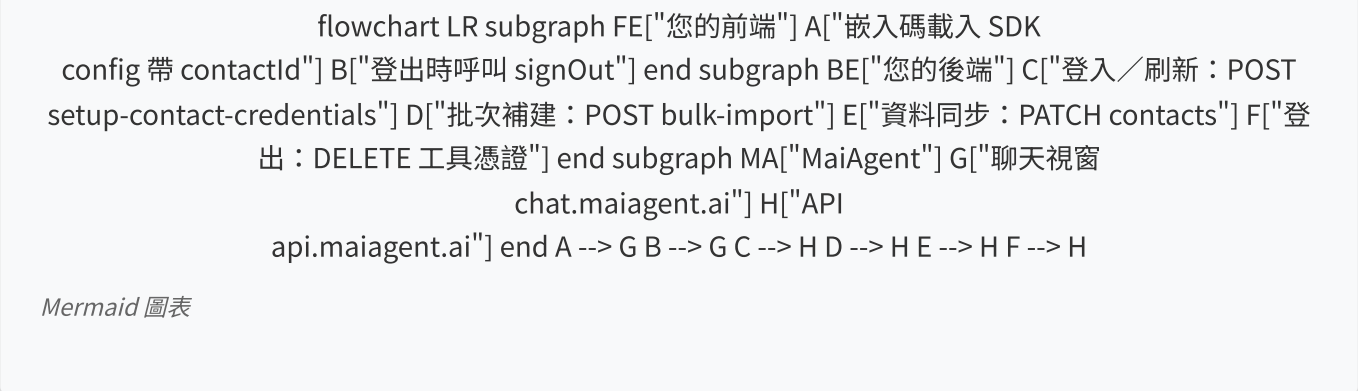
Mermaid 圖表

> 本頁的使用者是您的客戶（對應 MaiAgent 聯絡人）。若要串接的是貴公司或客戶的員工、要讓他們成為 MaiAgent 組織成員（用 MaiGPT、角色權限、Credits 額度），請看成員開通：SCIM／邀請／SSO。

核心概念只有一個：**聯絡人（Contact）**——您系統的每個使用者，對應到 MaiAgent 的一個聯絡人。聯絡人上掛著對話歷史、個人化屬性、知識庫查詢範圍與工具憑證；權限對接的所有動作，本質上都是在維護「您的使用者 ↔ 聯絡人」這條對應與其憑證。概念介紹見聯絡人 (Contact) 介紹及串接。

前後端各自負責什麼

整條旅程中，您的前端只負責「載入聊天視窗」，所有帶 Token 或 API Key 的呼叫都由您的後端發起：



Mermaid 圖表

其中 `setup-contact-credentials` 免 API Key（公開端點）；`bulk-import`、`PATCH`、`DELETE` 都需要 `Authorization: Api-Key` 認證，只能從後端呼叫。

步驟一：嵌入 Web Chat

> 先決定要嵌入哪個 AI Agent。 嵌入的能力（會不會生圖、上網、呼叫工具）取決於 Web Chat 背後的 Agent 設定，不取決於嵌入方式。組織內建的「MaiGPT」產品只供登入成員使用，其 Web Chat 不能嵌入外部系統；請為您的產品自建一個 AI Agent、設成 Agent 模式並掛上需要的工具，見串接 MaiGPT：選對 Agent、補齊能力。

依產品形態選擇嵌入方式，兩種用的是同一套 SDK 與同一套聯絡人機制：

形態	適合	文件
聊天泡泡（floating／sidebar）	客服、諮詢等輔助功能	Web Chat 嵌入與 SDK
MaiGPT 模式（類 ChatGPT 完整介面）	頁面主功能區塊、全站 AI 入口	MaiGPT 模式嵌入

> 要做身分對接的 Web Chat，建議此時就設定「允許嵌入網域」清單（管理者在後台對話平台的基本設定自行填寫），縮小可嵌入與可呼叫身分同步 API 的來源，見允許嵌入網域。

步驟二：首次整合——批次補建既有使用者

既有系統首次串接時，先把現有使用者的聯絡人一次補建起來，有兩條路：

方式	適合	文件
Excel 批次匯入（`POST /contacts/bulk-import/`）	大量使用者（上限 10,000 列／次），可同時帶入知識查詢範圍與 API 工具憑證	聯絡人批次匯入
逐筆呼叫身分同步 API	順著登入流程漸進補建、或量小	既有使用者批次補建

補建當下拿不到使用者的 Access Token 沒有關係——先建立對應即可，Token 憑證由步驟四的登入流程在各使用者下次登入時補上。

步驟三：新增使用者時建立聯絡人

補建完成後，在您系統的「建立帳號」流程中加一次 API 呼叫，讓新使用者一進來就有對應的聯絡人：

後端呼叫身分同步 API（`setup-contact-credentials`），以您系統的使用者識別碼作為 `sourceId` 把回傳的 `contactId` 存入會員資料表（建議欄位設計見登入流程整合位置）

`sourceId` 是冪等鍵：同一個 `sourceId` 重複呼叫只會更新、不會重複建立，所以這一步與步驟二、步驟四可以安全並存。

步驟四：登入與 Token 刷新

每次登入成功、產生 Access Token 後，後端再呼叫一次同一支身分同步 API，帶入**新的 Token**（`mcpCredentials`），讓 AI Agent 能以該使用者的身分與權限呼叫您系統的 API；Token 刷新時也是同一招——再呼叫一次即可，同一組（聯絡人、工具）為更新而非重複建立。完整序列圖與注意事項（非同步呼叫、失敗不阻擋登入）見聯絡人身分同步與 Token 更新。

前端則在頁面初始化 Web Chat 時帶入 `contactId`，見識別使用者。

> 若不需要讓 AI 呼叫您系統的 API、只要認得使用者，也可以完全不經後端：前端 config 帶 auth 即可，取捨見前端 auth 與後端串接怎麼選。

步驟五：登出與憑證註銷

登出時有兩件事，作用範圍不同、建議都做：

前端：呼叫 SDK 的 `signOut()`，聊天視窗回到匿名狀態

後端：刪除該聯絡人的工具憑證，讓 AI 即刻無法再以該使用者身分呼叫工具

詳細 API 與兩者的差異對照見登出與憑證註銷。

整合檢查清單

階段	動作	文件
建置	自建 AI Agent：Agent 模式＋掛上生圖等工具＋建立 Web Chat	串接 MaiGPT：選對 Agent、補齊能力
建置	嵌入 Web Chat（一般或 MaiGPT 模式）	Web Chat 嵌入與 SDK／MaiGPT 模式
建置	在後台對話平台的基本設定填寫「允許嵌入的網域」	允許嵌入網域
建置	會員資料表加 `maiagent_contact_id` 欄位	登入流程整合位置
建置	批次補建既有使用者	聯絡人批次匯入
營運	建立帳號流程呼叫身分同步 API	聯絡人身分同步與 Token 更新
營運	登入／Token 刷新流程更新憑證	呼叫時機

階段	動作	文件
營運	登出流程註銷憑證	登出與憑證註銷
進階	同步部門、會員等級等個人化屬性	同步聯絡人資料
進階	以查詢元資料限制各使用者的知識庫檢索範圍	知識管理權限（Query Metadata）

Web Chat 嵌入與 SDK

在網頁加入一段嵌入碼，即可將 MaiAgent 的 AI Agent 放進您的網站。嵌入碼載入 `embed.min.js`（以下稱 SDK）後，SDK 會在頁面上建立聊天按鈕與 iframe 聊天視窗，並提供全域 `MaiAgent` JavaScript API 供程式操作。

本頁是 Web Chat 嵌入的完整技術參考。相關主題：

串接 MaiGPT：選對 Agent、補齊能力 — MaiGPT 產品 vs MaiGPT 視窗模式，以及讓嵌入的 Agent 會生圖、會用工具

Web Chat MaiGPT 模式嵌入 — 類 ChatGPT 的全功能對話介面

Web Chat 注入頁面 Context — 把頁面資訊注入 LLM System Prompt

聯絡人身分同步與 Token 更新 — 後端建立聯絡人與憑證

軟體廠商整合指南 — 嵌入＋權限對接的完整旅程（批次補建、登入同步、登出註銷）

一、快速開始

在頁面 `</body>` 前加入：

```
<script>
  window.maiagentChatbotConfig = {
    webChatId: 'your-web-chat-id',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    primaryColor: '#1890ff',
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

`webChatId` 從管理後台的 Web Chat 設定頁取得（參考串接對話平台：網站，由後台產生的嵌入碼會自動帶入）

`baseUrl` 與 SDK 網址以 SaaS 環境（`chat.maiagent.ai`）為例；私有雲／地端部署請換成您環境的網域

`window.maiagentChatbotConfig` **必須在 SDK 腳本載入前定義**

預設行為：頁面右下角出現聊天按鈕，點擊開啟漂浮聊天視窗，使用者可在漂浮（floating）與側欄（sidebar）兩種視窗模式間切換。

初始化時機

SDK 載入後會註冊 `document.body.onload` 處理器執行初始化，一般情況不需手動介入。需要控制初始化時機時（例如 SPA 等容器 mount、或延遲渲染）：

延遲載入腳本：在適當時機才動態插入 `embed.min.js` 的